SPI EEPROM Motor-Profile Controller

In-Depth Technical Study Guide for Interviews and Viva

NXP LPC2138 (ARM7TDMI-S) · Microchip 25LC256 SPI EEPROM · TI L293D H-bridge · Bare-metal C and ARM assembly · Proteus VSM

How this guide is organised

Part	Contents	Use it for
1	Project summary, requirements, architecture	Opening answer, “explain your project”
2	ARM7TDMI core and LPC2138 microcontroller	Processor / architecture questions
3	SPI protocol and the LPC2138 SPI0 peripheral	Communication-protocol questions
4	25LC256 EEPROM: physics, protocol, driver	Memory questions
5	Profile format and data-integrity theory	“How do you know data is valid?”
6	Timers, PWM, DC motor and H-bridge theory	Motor-control and power questions
7	Toolchain, start-up code, linker script, memory map	Embedded-software fundamentals
8	Line-by-line firmware walkthrough	“Explain your code”
9	Proteus simulation, emulation and verification	Testing questions
10	Debugging case studies	“What problems did you face?”
11	Design alternatives and trade-offs	“Why did you choose X?”
12	Limitations and production roadmap	“What would you improve?”
13	Formula and numbers sheet	Last-minute revision
14	100 interview and viva questions with answers	Practice
15	Glossary and honesty notes	Final check

PART 1 — PROJECT SUMMARY AND ARCHITECTURE

1.1 Problem statement

Many machines repeat a fixed motion sequence: a conveyor that runs forward, pauses and reverses; a mixer that alternates direction; a pump that ramps up and holds. If the sequence is written into the program, changing it means editing code, rebuilding and reprogramming the microcontroller. If the sequence is held in RAM, it is lost at every power cut.

Goal: store the motion sequence (a *motor profile*) in non-volatile memory, read it reliably at power-up, protect it against corruption, and execute it safely on a DC motor.

1.2 The 60-second answer

> *"My project is an SPI EEPROM motor-profile controller built on the NXP LPC2138, an ARM7TDMI-S microcontroller. A motor profile — up to eight steps, each with a direction, a speed percentage, a ramp time and a run time — is stored in a Microchip 25LC256 SPI EEPROM so it survives power loss. At power-up the firmware reads the profile over SPI and validates it using magic bytes, a version number, range checks and a checksum. If the data is missing or corrupt, it writes a default profile, reads it back and validates it again, so the motor only ever runs data that really came from the EEPROM. The motor is driven through an L293D H-bridge: PWM on the enable pin sets the speed, two direction pins set the polarity, speed changes are ramped, and every reversal is preceded by a ramp-down and a 200 ms dead time. All the code is bare-metal C and ARM assembly — my own start-up file, linker script, SPI driver and EEPROM driver, with no vendor libraries. Every wait has a time-out and every failure produces an error code. I verified it in Proteus: the EEPROM self-test passed, the motor ran the profile in real time, and the EEPROM memory dump matches the stored profile byte for byte, including the checksum."*

1.3 Requirements

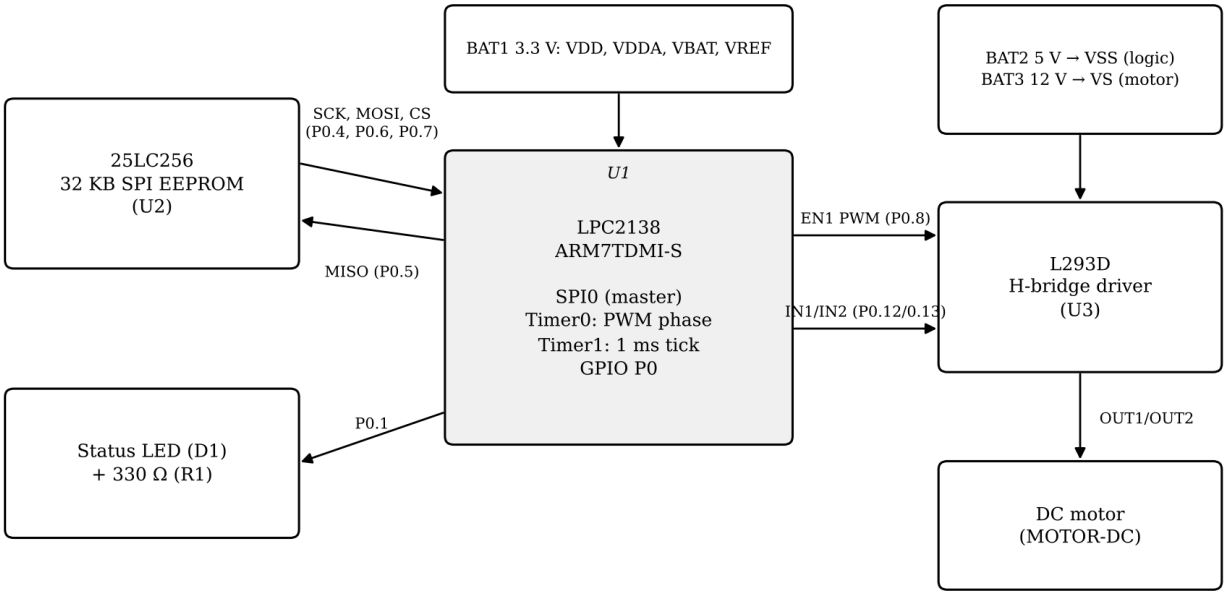
Functional requirements

ID	Requirement	How it is met
F1	Store a multi-step motor profile in non-volatile memory	25LC256 EEPROM, 29-byte record at address 0x0000
F2	Load and validate the profile at power-up	`profile_load()` → `profile_decode()`
F3	Recover automatically from a blank or corrupt EEPROM	Defaults written, read back and re-validated
F4	Control motor speed	100 Hz PWM on L293D EN1, 1 % resolution
F5	Control motor direction	L293D IN1/IN2
F6	Smooth speed changes	Linear ramps in 1 % steps
F7	Safe reversal	Ramp to 0 → coast → 200 ms → new direction
F8	Status indication	LED patterns and debug variables

Non-functional requirements

ID	Requirement	How it is met
N1	Never hang	Time-out on every SPI byte and every EEPROM write cycle
N2	Fail safe	Any error → motor disabled before anything else
N3	No dependencies	No HAL, no C library, no libgcc
N4	Debuggable in simulation	ELF with debug symbols; `g_*` variables in the Watch Window
N5	Simulate in real time	PWM frequency chosen so Proteus keeps up (CPU load 29 %)

1.4 System architecture



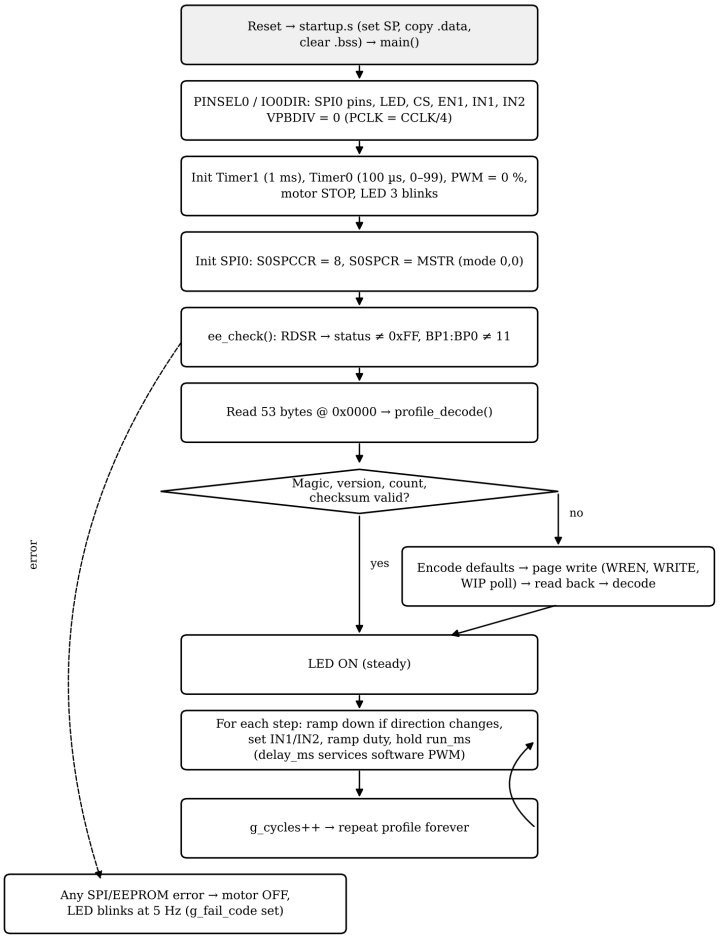
Subsystem	Component	Interface	Supply
Processing	LPC2138	—	3.3 V
Storage	25LC256	SPI0: SCK P0.4, MISO P0.5, MOSI P0.6, CS P0.7 (GPIO)	3.3 V
Actuation	L293D + DC motor	EN1 P0.8 (PWM), IN1 P0.12, IN2 P0.13	5 V logic, 12 V motor
Indication	LED + 330 Ω	P0.1	3.3 V

Three voltage domains: 3.3 V (logic), 5 V (driver logic), 12 V (motor power). They share a common ground so that logic levels are referenced to the same 0 V.

1.5 Firmware layers

Layer	Responsibility	Functions
Start-up	Stack, RAM initialisation, call `main`	`Reset_Handler` (`startup.s`)
Board support	Pins, clocks, timers	`timers_init`, `delay_ms`, `short_delay`
Driver: SPI	Byte exchange, chip select	`spi_transfer`, `cs_low`, `cs_high`
Driver: EEPROM	Status, write enable, page write, read	`ee_read_status`, `ee_wait_ready`, `ee_send_address`, `ee_write_page`, `ee_write_block`, `ee_read_block`, `ee_check`
Service: profile	Encode, decode, validate, load	`checksum`, `profile_encode`, `profile_decode`, `profile_load`
Service: motor	PWM, direction, ramps, steps	`pwm_init`, `pwm_service`, `motor_set_speed`, `motor_set_direction`, `motor_ramp`, `motor_run_step`
Application	Sequencing and error policy	`main`

Design principle: each layer only calls the layer below it. The motor code never touches SPI; the profile code never touches GPIO. This makes each part testable on its own.


Reset → startup.s (set SP, copy .data, clear .bss) → main()
PINSEL0 / IO0DIR: SPI0 pins, LED, CS, EN1, IN1, IN2
VPBDIV = 0 (PCLK = CCLK/4)
Init Timer1 (1 ms), Timer0 (100 µs, 0–99), PWM = 0 %, motor STOP, LED 3 blinks
Init SPI0: S0SPCCR = 8, S0SPCR = MSTR (mode 0,0)
ee_check(): RDSR → status ≠ 0xFF, BP1:BP0 ≠ 11
Read 53 bytes @ 0x0000 → profile_decode()
Magic, version, count, checksum valid?
no
yes
Encode defaults → page write (WREN, WRITE, WIP poll) → read back → decode
LED ON (steady)
For each step: ramp down if direction changes, set IN1/IN2, ramp duty, hold run_ms (delay_ms services software PWM)
g_cycles++ → repeat profile forever
error
Any SPI/EEPROM error → motor OFF, LED blinks at 5 Hz (g_fail_code set)

PART 2 — THE ARM7TDMI-S CORE AND THE LPC2138

2.1 What “ARM7TDMI-S” means

Letter	Meaning
ARM7	ARM processor family, ARMv4T architecture
T	**Thumb** — supports the 16-bit compressed Thumb instruction set as well as 32-bit ARM
D	**Debug** — on-chip debug support (halting, JTAG)
M	Enhanced **Multiplier** — 64-bit result multiply (`UMULL` , `SMULL`)
I	**EmbeddedICE** — hardware breakpoints and watchpoints
-S	**Synthesizable** — delivered as soft IP that chip makers synthesise into their silicon

2.2 Architecture

- **32-bit RISC**, load/store architecture: arithmetic works only on registers; memory is accessed only with `LDR` / `STR` .
- **Von Neumann**: a single bus for instructions and data (unlike the Harvard ARM9).
- **3-stage pipeline:** *fetch → decode → execute*. While one instruction executes, the next is decoded and the one after is fetched. Consequence: reading the PC in ARM state gives the current instruction address **+ 8**. This is why a branch offset is calculated relative to PC + 8.
- **No hardware divide instruction.** Division must be done in software. This is why the firmware contains `udiv()` .
- **Conditional execution:** almost every ARM instruction has a 4-bit condition field, so short `if` statements need no branch.

2.3 Registers and processor modes

The core has **37 registers**: 31 general-purpose 32-bit registers and 6 status registers. At any moment 16 general registers (R0–R15) and the CPSR are visible:

Register	Role
R0–R12	General purpose (R0–R3 carry function arguments and return values by convention)
R13 (SP)	Stack pointer
R14 (LR)	Link register — return address after `BL`
R15 (PC)	Program counter
CPSR	Current Program Status Register: flags N Z C V, interrupt masks I and F, Thumb bit T, mode bits M[4:0]
SPSR	Saved PSR — copy of the CPSR taken when an exception occurs (exception modes only)

Seven operating modes:

Mode	M[4:0]	Entered by	Banked registers
User	10000	Normal programs	—
FIQ	10001	Fast interrupt	R8–R14, SPSR
IRQ	10010	Normal interrupt	R13–R14, SPSR
Supervisor (SVC)	10011	**Reset**, SWI	R13–R14, SPSR
Abort	10111	Memory fault	R13–R14, SPSR
Undefined	11011	Unknown instruction	R13–R14, SPSR
System	11111	Privileged user code	— (shares User registers)

“Banked” means each mode has its own copy of those registers, so an exception handler gets a private stack pointer and link register without saving anything first. FIQ banks extra registers (R8–R12) to make it faster.

At reset the core is in **Supervisor mode**, in ARM state, with **IRQ and FIQ disabled** (I = F = 1). This project never leaves Supervisor mode and never enables interrupts, so **one stack** (set in `startup.s`) is enough.

2.4 Exception vector table

Address	Exception	Mode entered	Priority
0x00	Reset	SVC	1 (highest)
0x04	Undefined instruction	UND	6
0x08	Software interrupt (SWI)	SVC	6
0x0C	Prefetch abort	ABT	5
0x10	Data abort	ABT	2
0x14	**Reserved** — on LPC2000 holds the boot checksum	—	—
0x18	IRQ	IRQ	4
0x1C	FIQ	FIQ	3

Each entry is one 4-byte instruction, normally a branch to the real handler.

Decoding our actual vector table (from `objdump` of `firmware.elf`):

Address	Machine code	Instruction	How the target is computed
0x00	`EA000006`	`b Reset_Handler`	0x00 + 8 + 6×4 = **0x20**
0x04	`EA000017`	`b Undefined_Handler`	0x04 + 8 + 0x17×4 = **0x68**
…	`EA000017`	the other handlers	each entry is 4 bytes further on, as is each target

Encoding of `B`: bits 31–28 = condition (`1110` = always), bits 27–25 = `101` (branch), bit 24 = L (link, 0 for `B`), bits 23–0 = signed word offset. Range ±32 MB.

2.5 LPC2138 overview

Feature	LPC2138
Core	ARM7TDMI-S, up to 60 MHz
Flash	512 KB on-chip; the top part holds the NXP boot loader (about 500 KB usable)
SRAM	32 KB at 0x4000 0000
Package / GPIO	LQFP64; up to 47 GPIO, 5 V tolerant
Communication	2 UART, 2 I²C, SPI0, SSP (SPI1)
Timing	2 × 32-bit timers (4 match + 4 capture each), PWM unit (6 outputs), RTC, watchdog
Analog	2 × 10-bit ADC (8 channels each), 10-bit DAC
Interrupts	VIC — Vectored Interrupt Controller (16 vectored + non-vectored)
Programming	ISP over UART0 (boot loader), JTAG

2.6 Memory map

Range	Content
`0x0000 0000 – 0x0007 FFFF`	On-chip flash (our code starts at 0)
`0x4000 0000 – 0x4000 7FFF`	32 KB SRAM (our linker script uses the first 16 KB)
`0x7FFF D000 – …`	Boot block (boot loader)
`0xE000 0000 – 0xEFFF FFFF`	VPB peripherals (timers, UART, SPI, GPIO, pin connect, …)
`0xF000 0000 – 0xFFFF FFFF`	AHB peripherals (VIC at `0xFFFF F000`)

Peripheral addresses used by this project:

Peripheral	Base	Registers used
Timer0	`0xE000 4000`	T0TCR +0x04, T0TC +0x08, T0PR +0x0C, T0MCR +0x14, T0MR0 +0x18
Timer1	`0xE000 8000`	T1TCR, T1TC, T1PR, T1MCR (same offsets)
PWM	`0xE001 4000`	PWMTCR +0x04, PWMPR +0x0C, PWMMCR +0x14, PWMMR0 +0x18, PWMMR4 +0x40, PWMPCR +0x4C, PWMLER +0x50
SPI0	`0xE002 0000`	S0SPCR +0x00, S0SPSR +0x04, S0SPDR +0x08, S0SPCCR +0x0C
GPIO P0	`0xE002 8000`	IO0PIN +0x00, IO0SET +0x04, IO0DIR +0x08, IO0CLR +0x0C
Pin connect	`0xE002 C000`	PINSEL0 +0x00
System control	`0xE01F C000`	PCONP +0xC4, VPBDIV +0x100

How C reaches a register:

```
#define REG32(addr) (*(volatile unsigned int *)(addr))
#define IO0SET REG32(0xE0028004)
IO0SET = (1U << 1);   /* compiles to a 32-bit store (STR) to 0xE0028004 */
```

The address is cast to a pointer to a `volatile unsigned int` and dereferenced. `volatile` forces the compiler to perform every access exactly as written.

2.7 Boot process of a real LPC2138

1. After reset, the on-chip **boot loader** runs first.
2. If pin **P0.14 is low**, it enters **ISP mode** (In-System Programming over UART0 — used by Flash Magic).
3. Otherwise it checks for **valid user code**: the 32-bit sum of the eight vector-table words must be **0**. The reserved word at 0x14 is set to make this true.
4. If valid, it maps the user flash to address 0 (`MEMMAP` = 01) and jumps to the reset vector.

Our build: the vector words are `EA000006` followed by seven `EA000017`. For a valid sum the word at 0x14 would have to be **`0x99FFFF70`**. Our file leaves `EA000017` there, so on real hardware the programming tool (Flash Magic, `lpc21isp`) must insert the checksum — they do this automatically. Proteus does not check it.

2.8 Clock system

```
 Crystal (Fosc = 12 MHz) ──► [PLL, disabled] ──► CCLK = 12 MHz ──► [VPBDIV = /4] ──► PCLK = 3 MHz
                                                                                    │
 │
```

```
                                                ARM core                        Timers,
SPI0, PWM, UART
```

VPB divider (`VPBDIV`, bits 1:0): `00` → PCLK = CCLK/4 (reset value) · `01` → CCLK · `10` → CCLK/2.

PLL theory (not used here, but a common question): - CCLK = M × Fosc, where M = MSEL + 1 (1–32). - The internal current-controlled oscillator runs at F_CCO = CCLK × 2 × P, where P ∈ {1, 2, 4, 8}, and F_CCO must stay within **156–320 MHz**. - 60 MHz from 12 MHz: M = 5 → CCLK = 60 MHz; P = 2 → F_CCO = 60 × 2 × 2 = **240 MHz** ✓. - Programming sequence: write `PLLCFG`, enable in `PLLCON`, **feed sequence** `0xAA` then `0x55` to `PLLFEED` (protects against accidental changes), wait for the lock bit in `PLLSTAT`, then connect the PLL and feed again. - At high CCLK the **MAM** (Memory Accelerator Module) must be configured, because the flash is slower than the core; the MAM prefetches and buffers 128-bit flash lines.

Why this project runs at 12 MHz without the PLL: it keeps start-up simple and all the timing maths exact. The workload is tiny; even at 12 MHz the CPU spends most of its time waiting.

2.9 Pin connect block

Each port-0 pin can perform up to four functions, selected by two bits:

Register	Pins	Bits for pin n
PINSEL0	P0.0 – P0.15	2n + 1 : 2n
PINSEL1	P0.16 – P0.31	2(n−16) + 1 : 2(n−16)

Pin	PINSEL0 bits	00	01	10	11	Chosen
P0.1	3:2	GPIO	RxD0	PWM3	EINT0	00 GPIO (LED)
P0.4	9:8	GPIO	SCK0	CAP0.1	AD0.6	**01 SCK0**
P0.5	11:10	GPIO	MISO0	MAT0.1	AD0.7	**01 MISO0**
P0.6	13:12	GPIO	MOSI0	CAP0.2	AD1.0	**01 MOSI0**
P0.7	15:14	GPIO	SSEL0	PWM2	EINT2	**00 GPIO (CS)**
P0.8	17:16	GPIO	TxD1	PWM4	AD1.1	00 GPIO (software PWM) or 10 PWM4
P0.12, P0.13	25:24, 27:26	GPIO	…	…	…	00 GPIO (IN1, IN2)

PINSEL0 value = (1 << 8) | (1 << 10) | (1 << 12) = 0x100 + 0x400 + 0x1000 = **0x0000 1500**.

The firmware uses **read-modify-write** so other pins are untouched:

```
        PINSEL0 = (PINSEL0 & ~PINSEL0_MASK) | PINSEL0_VALUE;
```

2.10 GPIO

Register	Read	Write
IO0PIN	Current pin levels	Sets outputs directly (rarely used)
IO0SET	Output latch	1 → output high; 0 → no change
IO0CLR	—	1 → output low; 0 → no change
IO0DIR	Direction	1 → output; 0 → input

Why SET/CLR instead of read-modify-write on one register? `IO0SET = CS_PIN;` is a single store that affects only that pin. A read-modify-write (`IO0PIN |= x`) takes three instructions, and an interrupt between them could corrupt other pins. SET/CLR are therefore atomic and faster.

The firmware writes `IO0SET = CS_PIN` *before* making the pin an output, so the EEPROM chip select is high (inactive) from the very first moment it is driven, and no false transaction can start.

LED current: I = (3.3 V − V_F) / 330 Ω ≈ (3.3 − 2.0)/330 ≈ **3.9 mA** (assuming a typical red-LED forward voltage of about 2 V) — well within a GPIO pin's capability.

2.11 Power control (PCONP)

PCONP enables the clock to each peripheral; disabled peripherals consume less power. Bits used: PCTIM0 (1), PCTIM1 (2), PCPWM0 (5), PCSPI0 (8). These are on after reset, but the firmware sets them explicitly so it does not depend on the reset state.

PART 3 — SPI: THEORY AND THE LPC2138 SPI0 PERIPHERAL

3.1 What SPI is

Serial Peripheral Interface is a synchronous serial bus introduced by Motorola. Characteristics:

Property	SPI
Clocking	Synchronous — the master drives the clock (SCK)
Duplex	Full duplex — one bit in each direction on every clock
Topology	One master, one or more slaves
Slave selection	One chip-select (CS / SS) line per slave, active low
Addressing	None on the bus — CS chooses the device
Acknowledge	None — the protocol has no built-in error detection
Typical speed	Up to tens of MHz

Signals: SCK (clock, master → slave), MOSI (Master Out Slave In), MISO (Master In Slave Out), CS (chip select, master → slave). The 25LC256 calls them SCK, SI (= MOSI), SO (= MISO), CS.

3.2 How a byte moves — the shift-register model

Master and slave each contain an 8-bit shift register. The two are connected in a ring through MOSI and MISO. On every clock pulse, each side shifts one bit out and one bit in. After 8 clocks the two registers have **exchanged** their contents.

Consequences: 1. **Every transfer is an exchange.** Sending a byte always receives a byte (possibly meaningless). 2. **To read, send a dummy byte** (the firmware sends `0x00`) to generate the 8 clocks. 3. While sending a command, the byte received is ignored.

3.3 Clock polarity and phase — the four SPI modes

Mode	CPOL	CPHA	SCK idle	Data sampled on	Data changes on
0	0	0	Low	Rising edge	Falling edge
1	0	1	Low	Falling edge	Rising edge
2	1	0	High	Falling edge	Rising edge
3	1	1	High	Rising edge	Falling edge

The 25LC256 latches SI on the **rising** edge of SCK and drives SO on the **falling** edge, so it supports **mode 0 (0,0)** and **mode 3 (1,1)**. This project uses **mode 0**: CPOL = 0, CPHA = 0.

Mode 0 timing for one byte (MSB first):

```
CS   ‾‾\_____________________________________________/‾‾
SCK  ____/‾\_/‾\_/‾\_/‾\_/‾\_/‾\_/‾\_/‾\_______________
MOSI ===<b7><b6><b5><b4><b3><b2><b1><b0>===   (changes on falling edges)
            ↑ sampled on each rising edge
```

3.4 SPI compared with I²C and UART

Feature	SPI	I²C	UART
Wires	4 (+1 CS per extra slave)	2 (SDA, SCL)	2 (TX, RX)
Clock	Shared, from master	Shared, from master	None — both sides agree on a baud rate
Duplex	Full	Half	Full
Addressing	Chip-select line	7/10-bit address in the protocol	Point-to-point
Acknowledge	No	Yes (ACK/NACK bit)	No
Speed	Highest (MHz to tens of MHz)	100 k / 400 k / 1 M / 3.4 MHz	Typically ≤ 1 Mbaud
Pin cost for many devices	High	Low	—
Typical use	Flash, EEPROM, displays, ADCs	Sensors, EEPROMs, RTCs	Debug consoles, modules

Why SPI here? It is simple to drive at register level, full duplex, fast, and the 25LC256 is an SPI part. An I²C EEPROM (24LC256) would save two pins but needs addressing, ACK handling and open-drain pull-ups.

3.5 LPC2138 SPI0 registers in detail

S0SPCR — control register (`0xE002 0000`)

Bit	Name	Meaning	Our value
3	CPHA	Clock phase	0
4	CPOL	Clock polarity	0
5	**MSTR**	1 = master	**1**
6	LSBF	1 = LSB first	0 (MSB first, as the EEPROM requires)
7	SPIE	Interrupt enable	0 (polling)

→ `S0SPCR = 0x20`.

S0SPSR — status register (`0xE002 0004`)

Bit	Name	Meaning
3	ABRT	Slave abort (slave mode only)
4	**MODF**	**Mode fault** — SSEL went active while in master mode; the interface drops out of master mode
5	ROVR	Read overrun — a new byte arrived before the previous one was read
6	WCOL	Write collision — data register written during a transfer
7	**SPIF**	**Transfer complete**

SPIF clearing rule: SPIF is cleared by **reading S0SPSR and then accessing S0SPDR**. `spi_transfer()` does exactly this: it polls S0SPSR until SPIF is set, then reads S0SPDR.

S0SPDR — data register (`0xE002 0008`): writing it starts a transfer (master); reading it returns the received byte.

S0SPCCR — clock counter (`0xE002 000C`): SCK = PCLK / S0SPCCR. The value must be **even and ≥ 8**.

Our clock: SCK = 3 MHz / 8 = **375 kHz** → bit time 2.67 µs → **byte time ≈ 21.3 µs**. This is the fastest SPI0 can go at PCLK = 3 MHz. The EEPROM allows MHz-range clocks, so there is a large margin; the slow clock also makes waveforms easy to see in simulation.

3.6 Why the chip select is a GPIO pin

On the LPC2138, P0.7 can be configured as **SSEL0**. In master mode SSEL0 is an **input**: if anything pulls it low, the SPI raises a **mode fault (MODF)** and leaves master mode — a feature intended for multi-master systems. It is not a chip-select output.

The EEPROM also needs CS held low across a whole multi-byte command (opcode, two address bytes, then data). So P0.7 is configured as a plain **GPIO output** (`PINSEL0` bits 15:14 = 00) and driven by software:

```
static void cs_low(void)  { IO0CLR = CS_PIN; short_delay(); }
static void cs_high(void) { short_delay(); IO0SET = CS_PIN; short_delay(); }
```

The short delays (a 10-iteration loop, a few microseconds at 12 MHz) guarantee the EEPROM's CS setup, hold and deselect times, which are in the tens to hundreds of nanoseconds.

The original bug: `PINSEL0 = 0x5500` set bits 15:14 to 01, selecting SSEL0, so writes to `IO0CLR` for P0.7 had no effect on the pin. The fix was `0x1500`.

3.7 The transfer function — design for robustness

```
/* Send/receive one byte; returns an error on time-out or fault */
static unsigned int spi_transfer(unsigned char tx, unsigned char *rx)
{
```

```
    unsigned int timeout = SPI_TIMEOUT;
    unsigned int status;
    unsigned char data;

    S0SPDR = tx;

    for (;;) {
        status = S0SPSR;

        if ((status & SPSR_SPIF) != 0U) {
            break;
        }
        if ((status & (SPSR_MODF | SPSR_ABRT)) != 0U) {
            return FAIL_SPI_FAULT;
        }
        if (--timeout == 0U) {
            return FAIL_SPI_TIMEOUT;
        }
    }

    data = (unsigned char)S0SPDR;

    if (rx != 0) {
        *rx = data;
    }

    return FAIL_NONE;
}
```

Line(s)	Purpose
`S0SPDR = tx;`	Starts the 8-clock exchange
`status = S0SPSR;`	Reading the status is also step 1 of clearing SPIF
`SPSR_SPIF` set → break	Transfer finished
`SPSR_MODF` or `SPSR_ABRT` → `FAIL_SPI_FAULT` (2)	Hardware fault; continuing would be meaningless
`--timeout == 0` → `FAIL_SPI_TIMEOUT` (1)	SPI never finished — wrong pin-mux, clock off, broken model
`data = S0SPDR;`	Step 2 of clearing SPIF; returns the received byte

Time-out sizing: a byte takes 8 × 8 = 64 PCLK = 256 CCLK cycles. Each poll iteration costs on the order of 10–20 CCLK cycles at -O0, so SPIF normally appears within about 20 polls. The limit of 100 000 polls is thousands of times longer: it never triggers by accident, yet a fault is reported within tens of milliseconds instead of hanging forever.

PART 4 — THE 25LC256 SPI EEPROM

4.1 How an EEPROM cell stores data

An EEPROM (Electrically Erasable Programmable Read-Only Memory) bit is a **floating-gate MOSFET**: a transistor with an extra gate completely surrounded by insulating oxide.

- **Programming / erasing:** a high voltage (generated on-chip by a **charge pump**, typically 10–20 V) across a very thin oxide lets electrons **tunnel** through it (**Fowler–Nordheim tunnelling**) onto or off the floating gate.
- **Reading:** charge on the floating gate shifts the transistor's threshold voltage; a sense amplifier detects whether the transistor conducts at the read voltage → 0 or 1.
- **Retention:** the trapped charge has no conductive path to leak through, so data is kept for decades without power.
- **Wear:** every program/erase cycle damages the oxide slightly (trapped charge, defects). After a large number of cycles the cell no longer holds charge reliably. This is the **endurance** limit — about **1 million** cycles per byte for the 25LC256.
- **Why writes take milliseconds:** tunnelling is slow and the device first erases, then programs, then verifies, all self-timed. Reads take nanoseconds.

EEPROM vs flash: both use floating gates. EEPROM erases and writes **individual bytes**; flash erases large **sectors** at once, which is denser and cheaper but needs sector management. That is why small, frequently updated parameters often go in EEPROM.

4.2 25LC256 key specifications

Parameter	Value
Organisation	32 768 × 8 bits (256 Kbit = 32 KB)
Page size	64 bytes
Supply	2.5–5.5 V (25AA256 variant: 1.8–5.5 V)
SPI modes	0,0 and 1,1
Max clock	Up to 10 MHz at higher supply voltages; lower at reduced VCC (see datasheet)
Write cycle t_WC	Self-timed, **5 ms maximum**
Endurance	About 1 000 000 erase/write cycles
Retention	> 200 years (datasheet figure)
Address	16-bit; the top bit is ignored (15 bits address 32 KB)

Pinout (DIP/SOIC-8): 1 CS · 2 SO · 3 WP · 4 VSS · 5 SI · 6 SCK · 7 HOLD · 8 VCC.

- **WP (write protect):** when the WPEN bit is set and WP is low, writes to the *status register* are blocked. Tied high here.
- **HOLD:** pulling HOLD low (while SCK is low) pauses a transfer so the master can service another device; releasing it resumes. Tied high here.

4.3 Instruction set

Instruction	Opcode	Sequence	Notes
READ	0x03	CS↓, 03, A15–A8, A7–A0, then clock out data… CS↑	Address auto-increments; reads can run through the whole array
WRITE	0x02	CS↓, 02, A15–A8, A7–A0, D0 … D63, CS↑	Up to 64 bytes in one page; **write starts at CS↑**
WRDI	0x04	CS↓, 04, CS↑	Clears WEL
WREN	**0x06**	CS↓, 06, **CS↑**	Sets WEL; takes effect only when CS returns high
RDSR	**0x05**	CS↓, 05, read status, CS↑	Allowed even during a write cycle
WRSR	0x01	CS↓, 01, value, CS↑	Writes BP0, BP1, WPEN

4.4 Status register

Bit	Name	Meaning
0	**WIP**	Write In Progress — 1 while the internal write cycle runs (read-only)
1	**WEL**	Write Enable Latch — must be 1 for WRITE/WRSR (read-only, set by WREN)
2	BP0	Block protect bit 0
3	BP1	Block protect bit 1
7	WPEN	Write-protect enable (works with the WP pin)

Block protection:

BP1 BP0	Protected range
0 0	None
0 1	Upper quarter (0x6000–0x7FFF)
1 0	Upper half (0x4000–0x7FFF)
1 1	Entire array (0x0000–0x7FFF)

`ee_check()` treats BP1:BP0 = 11 as error 4, because the profile at 0x0000 could not be written.

WEL is cleared automatically at power-up, after WRDI, and after every completed WRITE or WRSR. So **WREN is needed before every single write operation**.

4.5 Page writes and the page-wrap trap

The array is divided into 64-byte pages: 0x0000–0x003F, 0x0040–0x007F, and so on. One WRITE command can store 1 to 64 bytes, but only **within one page**. If the data runs past the end of the page, the internal address **wraps to the start of the same page** and overwrites earlier bytes — a silent corruption.

Rule: split every block write at page boundaries. `ee_write_block()` computes:

```
chunk = 64 − (address mod 64)        // bytes left in this page
if chunk > remaining: chunk = remaining
```

Example: writing 10 bytes starting at 0x003C → first chunk = 64 − 60 = 4 bytes (0x3C–0x3F), second chunk = 6 bytes (0x40–0x45), each with its own WREN, WRITE and WIP wait.

Our 29-byte profile starts at 0x0000, so it fits in one page and is written with a single WRITE command.

4.6 Why poll WIP instead of waiting a fixed 5 ms

Fixed delay	WIP polling
Always waits the worst case	Finishes as soon as the write is done (often well under 5 ms)
Depends on the CPU clock being right	Independent of CPU speed
Cannot detect a dead or stuck device	A time-out on polling detects it (error 6)

The original test code used a 100 000-iteration delay loop; the final driver polls RDSR up to 20 000 times.

4.7 The EEPROM driver, step by step

Reading the status register:

```
        static unsigned int ee_read_status(unsigned char *status)
        {
            unsigned int err;

            cs_low();
            err = spi_transfer(EE_RDSR, 0);
            if (err == FAIL_NONE) {
                err = spi_transfer(0x00U, status);
            }
            cs_high();

            return err;
        }
```

Waiting for a write cycle to finish:

```
        static unsigned int ee_wait_ready(void)
        {
            unsigned int polls;
```

```
    unsigned int err;
    unsigned char status;

    for (polls = 0; polls < WIP_TIMEOUT; polls++) {
        err = ee_read_status(&status);
        if (err != FAIL_NONE) {
            return err;
        }
        if ((status & SR_WIP) == 0U) {
            return FAIL_NONE;
        }
    }

    return FAIL_WRITE_TIMEOUT;
}
```

Writing one page (the core of the driver):

```
/* WREN, confirm WEL, WRITE, then wait for WIP to clear */
static unsigned int ee_write_page(unsigned int address,
                                  const unsigned char *buf, unsigned int len)
{
    unsigned int err;
    unsigned int i;
    unsigned char status = 0U;

    cs_low();
    err = spi_transfer(EE_WREN, 0);
    cs_high();
    if (err != FAIL_NONE) {
        return err;
    }

    err = ee_read_status(&status);
    if (err != FAIL_NONE) {
        return err;
    }
    if ((status & SR_WEL) == 0U) {
        return FAIL_WEL_NOT_SET;
    }

    cs_low();
    err = ee_send_address(EE_WRITE, address);
    for (i = 0; (i < len) && (err == FAIL_NONE); i++) {
        err = spi_transfer(buf[i], 0);
    }
    cs_high();

    if (err != FAIL_NONE) {
        return err;
    }

    return ee_wait_ready();
}
```

Sequence on the bus for writing the 29-byte profile:

```
CS↓ 06 CS↑                                  WREN
CS↓ 05 xx→02 CS↑                            RDSR: WEL = 1 ✓
CS↓ 02 00 00 4D 50 02 04 01 4B … 47 CS↑     WRITE 29 bytes at 0x0000 (write starts at
CS↑)
```

```
CS↓ 05 xx→03 CS↑   (repeated)              RDSR: WIP = 1, WEL = 1 (busy)
CS↓ 05 xx→00 CS↑                           RDSR: WIP = 0, WEL = 0 → done
```

Why check WEL after WREN? If MISO were stuck low, every status read would return 0x00, which looks like “not busy” — the firmware would believe every write succeeded. Requiring WEL = 1 proves that the EEPROM received the WREN command **and** that data really comes back on MISO. Error 5 catches both an open SI line and a stuck SO line.

Splitting at page boundaries:

```
        /* Split writes at 64-byte page boundaries */
        static unsigned int ee_write_block(unsigned int address,
                                           const unsigned char *buf, unsigned int len)
        {
            unsigned int err;
            unsigned int chunk;

            while (len > 0U) {
                chunk = EE_PAGE_SIZE - (address & (EE_PAGE_SIZE - 1U));
                if (chunk > len) {
                    chunk = len;
                }

                err = ee_write_page(address, buf, chunk);
                if (err != FAIL_NONE) {
                    return err;
                }

                address += chunk;
                buf += chunk;
                len -= chunk;
            }

            return FAIL_NONE;
        }
```

Sequential read:

```
        static unsigned int ee_read_block(unsigned int address,
                                          unsigned char *buf, unsigned int len)
        {
            unsigned int err;
            unsigned int i;

            cs_low();
            err = ee_send_address(EE_READ, address);
            for (i = 0; (i < len) && (err == FAIL_NONE); i++) {
                err = spi_transfer(0x00U, &buf[i]);
            }
            cs_high();

            return err;
        }
```

Reading the 53-byte profile window costs 3 + 53 = 56 bytes × 21.3 µs ≈ **1.2 ms** of bus time.

Presence check before trusting any data:

```
        static unsigned int ee_check(void)
        {
```

```
            unsigned int err;
            unsigned char status = 0U;

            err = ee_read_status(&status);
            if (err != FAIL_NONE) {
                return err;
            }
            if (status == 0xFFU) {
                return FAIL_NO_RESPONSE;
            }
            if ((status & SR_BP) == SR_BP) {
                return FAIL_PROTECTED;
            }
            return FAIL_NONE;
        }
```

A status of **0xFF** means nothing is driving MISO (EEPROM unpowered, SO disconnected, CS not reaching the chip): the line floats or is pulled high. A real status can never be 0xFF, because bits 4–6 always read 0.

PART 5 — PROFILE FORMAT AND DATA-INTEGRITY THEORY

5.1 What a profile is

A profile is an ordered list of up to 8 **steps**. Each step has four fields:

Field	Type	Range	Meaning
direction	8-bit	0 = stop, 1 = forward, 2 = reverse	Motor direction
speed	8-bit	0–100	PWM duty in percent
ramp_ms	16-bit	0–65 535	Time to reach the new speed
run_ms	16-bit	0–65 535	Time to hold that speed

In C:

```
typedef struct {
    unsigned char  direction;
    unsigned char  speed;
    unsigned short ramp_ms;
    unsigned short run_ms;
} motor_step_t;
```

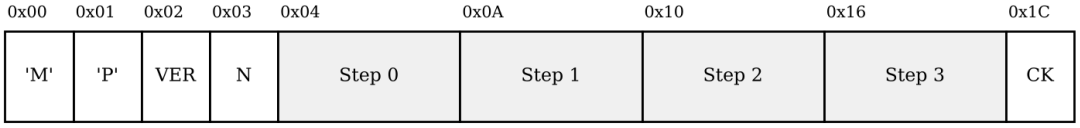
5.2 Serialisation — why not store the struct directly?

Writing the struct's raw memory to the EEPROM would be fragile:

- **Padding:** compilers may insert gaps for alignment, which change between compilers and settings.
- **Endianness:** the ARM stores multi-byte numbers little-endian; another tool reading the EEPROM might assume big-endian.
- **Version drift:** if the struct changes, old data would be misread.

So `profile_encode()` writes every field explicitly, byte by byte, with 16-bit values **big-endian** (high byte first). `profile_decode()` does the reverse. The on-EEPROM format is therefore defined by the code, not by the compiler.

5.3 Byte layout



Each step (6 bytes): direction | speed (%) | ramp_ms[15:8] | ramp_ms[7:0] | run_ms[15:8] | run_ms[7:0]

VER = PROFILE_VERSION, N = number of steps, CK = checksum chosen so that the sum of all 29 bytes ≡ 0 (mod 256)

Offset	Size	Field	Value in default profile
0x00	1	Magic 0	`'M'` = 0x4D
0x01	1	Magic 1	`'P'` = 0x50
0x02	1	Version	0x02
0x03	1	Step count N	0x04
0x04 + 6k	6	Step k	see below
0x04 + 6N	1	Checksum	0x47

Size = 4 + 6N + 1 bytes → **29 bytes** for N = 4, **53 bytes** maximum for N = 8. Both fit in the first 64-byte page.

Encoding the default profile:

Step	Direction	Speed	ramp_ms	run_ms	Bytes
0	1 Forward	75 = 0x4B	500 = 0x01F4	3000 = 0x0BB8	`01 4B 01 F4 0B B8`
1	0 Stop	0	300 = 0x012C	1000 = 0x03E8	`00 00 01 2C 03 E8`
2	2 Reverse	40 = 0x28	500	3000	`02 28 01 F4 0B B8`
3	0 Stop	0	300	1000	`00 00 01 2C 03 E8`

```
        static unsigned int profile_encode(const motor_step_t *src, unsigned int count,
                                           unsigned char *buf)
        {
            unsigned int i;
            unsigned int n = HEADER_BYTES;

            buf[0] = PROFILE_MAGIC_0;
            buf[1] = PROFILE_MAGIC_1;
            buf[2] = PROFILE_VERSION;
            buf[3] = (unsigned char)count;

            for (i = 0; i < count; i++) {
                buf[n++] = src[i].direction;
                buf[n++] = src[i].speed;
                buf[n++] = (unsigned char)(src[i].ramp_ms >> 8);
                buf[n++] = (unsigned char)(src[i].ramp_ms & 0xFFU);
```

```
            buf[n++] = (unsigned char)(src[i].run_ms >> 8);
            buf[n++] = (unsigned char)(src[i].run_ms & 0xFFU);
        }

        buf[n] = checksum(buf, n);
        return n + 1U;
    }
```

5.4 The checksum — maths and properties

```
    static unsigned char checksum(const unsigned char *buf, unsigned int len)
    {
        unsigned int i;
        unsigned char sum = 0U;

        for (i = 0; i < len; i++) {
            sum = (unsigned char)(sum + buf[i]);
        }
        return (unsigned char)(0U - sum);
    }
```

The stored byte is **C = (−S) mod 256**, where S is the sum of all preceding bytes. Therefore the sum of **all** bytes including C is **0 mod 256**, and the reader simply checks that the total is zero.

Worked example with the real EEPROM contents:

Part	Bytes	Sum
Header	4D 50 02 04	77 + 80 + 2 + 4 = 163
Step 0	01 4B 01 F4 0B B8	1 + 75 + 1 + 244 + 11 + 184 = 516
Step 1	00 00 01 2C 03 E8	0 + 0 + 1 + 44 + 3 + 232 = 280
Step 2	02 28 01 F4 0B B8	2 + 40 + 1 + 244 + 11 + 184 = 482
Step 3	00 00 01 2C 03 E8	280
Total S		**1721**

1721 mod 256 = 185 (1721 = 6 × 256 + 185) → C = 256 − 185 = **71 = 0x47**. ✓ This matches the Proteus memory dump.

What an 8-bit additive checksum detects and misses:

Error	Detected?
Any single corrupted byte	**Always** — the sum changes by a non-zero amount below 256
A whole region erased to 0xFF	Almost always
Two errors that cancel (e.g. +5 in one byte, −5 in another)	No
Two bytes swapped	**No** — addition is order-independent
Errors totalling exactly 256	No

CRC comparison: a CRC (cyclic redundancy check) treats the data as a polynomial and divides it by a generator polynomial; the remainder is the check value. A CRC-16 detects all single- and

double-bit errors, all odd numbers of bit errors (for suitable polynomials), all burst errors up to 16 bits long, and byte swaps. It costs a little more code and time. For a production system a CRC-16 is the better choice; the additive checksum was chosen here for simplicity and transparency.

5.5 Validation — `profile_decode()`

```
static unsigned int profile_decode(const unsigned char *buf)
{
    unsigned int count = buf[3];
    unsigned int len;
    unsigned int i;
    unsigned int n = HEADER_BYTES;
    unsigned char sum = 0U;

    if ((buf[0] != PROFILE_MAGIC_0) || (buf[1] != PROFILE_MAGIC_1) ||
        (buf[2] != PROFILE_VERSION) || (count == 0U) || (count > MAX_STEPS)) {
        return 0U;
    }

    len = HEADER_BYTES + (count * STEP_BYTES) + 1U;
    for (i = 0; i < len; i++) {
        sum = (unsigned char)(sum + buf[i]);
    }
    if (sum != 0U) {
        return 0U;
    }

    for (i = 0; i < count; i++) {
        if ((buf[n] > DIR_REVERSE) || (buf[n + 1U] > 100U)) {
            return 0U;
        }
        steps[i].direction = buf[n];
        steps[i].speed     = buf[n + 1U];
        steps[i].ramp_ms   = (unsigned short)((buf[n + 2U] << 8) | buf[n + 3U]);
        steps[i].run_ms    = (unsigned short)((buf[n + 4U] << 8) | buf[n + 5U]);
        n += STEP_BYTES;
    }

    step_count = count;
    return 1U;
}
```

Validation order and the reason for each check:

1. **Magic** `'M' 'P'` — a blank EEPROM reads all 0xFF; random data almost never starts with these two bytes.
2. **Version =** `PROFILE_VERSION` — rejects data written by older firmware with a different meaning.
3. **1 ≤ N ≤ 8** — protects against reading past the 53-byte buffer and the `steps[]` array (a memory-safety check).
4. **Checksum = 0** — detects corruption.
5. **Range checks** (direction ≤ 2, speed ≤ 100) — even if the checksum passes, never run an impossible value.

The checks are ordered from cheapest to most expensive, and the length check comes **before** the checksum loop, so the loop never reads outside the buffer.

5.6 Loading and self-healing — `profile_load()`

```c
/* Load the profile; if invalid, store defaults and read them back */
static unsigned int profile_load(void)
{
    unsigned char buf[PROFILE_MAX_BYTES];
    unsigned int len;
    unsigned int err;

    err = ee_check();
    if (err != FAIL_NONE) {
        return err;
    }

    err = ee_read_block(PROFILE_ADDR, buf, PROFILE_MAX_BYTES);
    if (err != FAIL_NONE) {
        return err;
    }
    if (profile_decode(buf) != 0U) {
        g_profile_source = SOURCE_EEPROM;
        return FAIL_NONE;
    }

    len = profile_encode(default_steps, DEFAULT_STEP_COUNT, buf);
    err = ee_write_block(PROFILE_ADDR, buf, len);
    if (err != FAIL_NONE) {
        return err;
    }

    err = ee_read_block(PROFILE_ADDR, buf, PROFILE_MAX_BYTES);
    if (err != FAIL_NONE) {
        return err;
    }
    if (profile_decode(buf) == 0U) {
        return FAIL_PROFILE_BAD;
    }

    g_profile_source = SOURCE_DEFAULTS;
    return FAIL_NONE;
}
```

Situation	What happens	`g_profile_source`
Valid profile present	Used as-is; nothing written	1
Blank (0xFF), corrupt or old version	Defaults encoded → written → **read back** → validated	2
EEPROM not answering / write fails	Error code returned; motor never starts	—

Why read back after writing the defaults? It proves the write really happened and that the data survives the round trip. The motor therefore only ever runs data that has come *out of* the EEPROM, never directly from the program constants.

Write-endurance impact: the EEPROM is written only when the stored profile is invalid — normally once in the product's life. With about 1 million cycles of endurance, wear is not a concern.

5.7 Power-loss and atomicity (an advanced viva topic)

If power fails *during* the 5 ms write cycle, the page may be left partly written: a **torn write**. This design detects it at the next boot (checksum fails) and restores the defaults — safe, but the user's custom profile would be lost. A production design would keep **two copies (A/B)**, each with a sequence counter and a CRC, write the inactive copy first, and switch to it only after it verifies. At boot it chooses the newest valid copy, so there is always one good profile.

PART 6 — TIMERS, PWM, DC MOTOR AND H-BRIDGE

6.1 LPC2138 timer architecture

Each 32-bit timer contains:

Register	Function
TCR	Timer Control: bit 0 = enable, bit 1 = reset
PR	Prescale Register: the prescale counter (PC) counts PCLK cycles up to PR
PC	Prescale Counter
TC	**Timer Counter**: increments once every **PR + 1** PCLK cycles
MR0–MR3	Match registers, compared with TC
MCR	Match Control: for each MRx, 3 bits = interrupt / **reset TC** / stop on match
CR0–CR3, CCR	Capture registers and control (not used)

Tick period = (PR + 1) / PCLK.

Timer1 — the millisecond clock - PR = PCLK/1000 − 1 = 3 000 000/1000 − 1 = **2999** → TC increases every 1 ms. - `delay_ms(ms)` records `start = T1TC` and waits until `T1TC − start ≥ ms`. Unsigned subtraction works correctly even when TC wraps around (after about 49.7 days).

Timer0 — the PWM phase counter - PR = PCLK/10 000 − 1 = **299** → TC increases every **100 µs**. - MR0 = 99 with the “reset on MR0” bit set in MCR (bit 1) → TC counts **0, 1, …, 99, 0, …** → period = 100 × 100 µs = **10 ms**.

```
        static void timers_init(void)
        {
            PCONP |= PCONP_PCTIM0 | PCONP_PCTIM1;

            T1TCR = TCR_RESET;
            T1PR = (PCLK_HZ / 1000U) - 1U;
            T1MCR = 0U;
            T1TCR = TCR_ENABLE;

            T0TCR = TCR_RESET;
            T0PR = (PCLK_HZ / PWM_TICK_HZ) - 1U;
            T0MR0 = PWM_STEPS - 1U;
            T0MCR = MCR_MR0_RESET;
            T0TCR = TCR_ENABLE;
        }
```

6.2 PWM fundamentals

Pulse-width modulation switches a signal fully on and fully off at a fixed frequency; the **duty cycle** D is the fraction of each period spent on:

D = t_on / T, V_avg = D × V_supply

Duty	Average voltage (12 V supply, ideal)
40 %	4.8 V
75 %	9.0 V
100 %	12 V

Why PWM instead of a variable voltage? The transistors are either fully on (small voltage drop) or fully off (no current), so little power is wasted as heat, compared with a linear regulator dropping the excess voltage.

6.3 Software PWM — how it works here

```
/* Software PWM: EN1 high while Timer0 count < duty */
static void pwm_service(void)
{
#if !USE_HW_PWM
    unsigned int want_high;

    if (pwm_duty == 0U) {
        want_high = 0U;
    } else if (pwm_duty >= PWM_STEPS) {
        want_high = 1U;
    } else {
        want_high = (T0TC < pwm_duty) ? 1U : 0U;
    }

    if (want_high != en1_is_high) {
        if (want_high != 0U) {
            IO0SET = EN1_PIN;
        } else {
            IO0CLR = EN1_PIN;
        }
        en1_is_high = want_high;
    }
#endif
}
```

- Timer0 is a free-running 0–99 ramp. EN1 is driven high while T0TC < duty, low otherwise → the high time is `duty × 100 µs` out of every 10 ms.
- Special cases: duty 0 → always low; duty ≥ 100 → always high (avoids a glitch at the period boundary).
- The pin is only written when its required state **changes**, which minimises bus traffic and simulated edges.
- `pwm_service()` is called continuously inside `delay_ms()`. Since the program spends almost all its time inside `delay_ms()`, the PWM runs continuously. During EEPROM access at start-up the duty is 0, so nothing is lost.

```
static void delay_ms(unsigned int ms)
{
    unsigned int start = T1TC;

    while ((T1TC - start) < ms) {
        pwm_service();
```

```
        }
    }
```

Resolution: 100 steps → **1 %**. **Jitter:** an edge can be late by one polling-loop iteration (a few µs), negligible against a 100 µs step.

6.4 Hardware PWM — the alternative (`USE_HW_PWM 1`)

The LPC2138 PWM unit is a timer with seven match registers:

- **PWMMR0** sets the period (TC resets at MR0).
- In **single-edge mode**, output PWMn is **set** at the start of each period and **reset** when TC matches **MRn**.
- **PWMLER** (latch enable): new match values are copied in at the start of the next period, so the duty changes without glitches.
- **PWMPCR** enables each output (bit 12 = PWM4 on P0.8).
- **PWMTCR**: bit 0 counter enable, bit 1 reset, bit 3 PWM mode enable.

```
static void pwm_init(void)
{
#if USE_HW_PWM
    PCONP |= PCONP_PCPWM0;
    PWMTCR = PWMTCR_COUNTER_RESET;
    PWMPR = (PCLK_HZ / PWM_TICK_HZ) - 1U;
    PWMMR0 = PWM_STEPS;
    PWMMR4 = 0U;
    PWMMCR = PWMMCR_RESET_ON_MR0;
    PWMPCR = PWMPCR_ENA4;
    PWMLER = PWMLER_EN0 | PWMLER_EN4;
    PWMTCR = PWMTCR_COUNTER_ENABLE | PWMTCR_PWM_ENABLE;
#else
    IO0CLR = EN1_PIN;
    en1_is_high = 0U;
#endif
    pwm_duty = 0U;
}
```

```
static void motor_set_speed(unsigned int percent)
{
    if (percent > 100U) {
        percent = 100U;
    }

#if USE_HW_PWM
    if (percent >= 100U) {
        PWMMR4 = PWM_STEPS + 1U;
    } else {
        PWMMR4 = percent;
    }
    PWMLER = PWMLER_EN4;
#endif

    pwm_duty = percent;
    g_speed = percent;
    pwm_service();
}
```

Hardware PWM needs no CPU time and has zero jitter. On real hardware it is the better choice, at **≥ 20 kHz** (above hearing, smoother motor current).

6.5 Why 100 Hz in this project — the simulation-speed problem

Proteus simulates the microcontroller digitally, but the **L293D and the motor are analog models** solved by a SPICE-like engine. Every PWM edge forces the analog solver to shrink its time step and recompute the circuit around the edge.

PWM	Edges per second	Proteus behaviour
3 kHz (first version)	6000	CPU load > 100 %; less than 1 s simulated after a long wait; motor appeared stuck at 0 RPM
100 Hz (final)	≤ 200 (≈ 142 measured)	**CPU load 29 %**, real time; motor runs

The fix was diagnostic, not cosmetic: the pin indicators showed IN1 already high (so the profile had loaded) and the status bar showed the simulation crawling — the firmware was fine; the simulator was the bottleneck.

6.6 DC motor theory

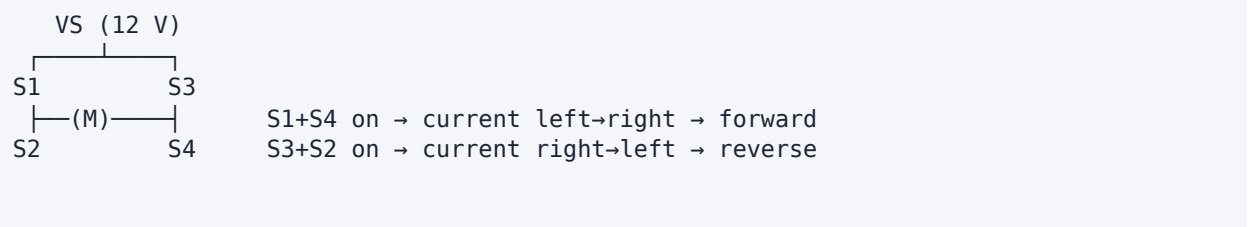
A brushed DC motor is modelled by:

- **Electrical:** $V = R \cdot i + L \cdot (di/dt) + E$, where the back-EMF $E = K_e \cdot \omega$
- **Mechanical:** torque $T = K_t \cdot i$; $J \cdot (d\omega/dt) = T - T_{load} - B \cdot \omega$

Consequences: 1. **Steady-state speed ≈ (V − R·i) / K_e** → speed rises roughly linearly with average voltage, i.e. with PWM duty. 2. **Starting current** (ω = 0, so E = 0): i = V/R — the stall current, much higher than the running current. **Ramping limits this inrush.** 3. **Instant reversal:** the motor is still spinning, so E still opposes the old supply polarity. When the polarity flips, E *adds* to the new supply voltage: i ≈ (V + E)/R — up to **twice the stall current**. This is why the firmware ramps down and pauses before reversing. 4. **Electrical time constant** $\tau_e = L/R$. If the PWM period is much shorter than τ_e, the current stays almost constant through each period. At 100 Hz the period (10 ms) is longer than a typical τ_e, so the current pulses, but the **mechanical** time constant (inertia, typically tens to hundreds of milliseconds) still smooths the speed. On hardware, 20 kHz gives smoother current, less heating and no audible whine.

6.7 The H-bridge and the L293D

An **H-bridge** places the motor between two half-bridges; turning on opposite corners connects the motor one way or the other:



```
                                  S1+S2 (or S3+S4) on together → SHORT CIRCUIT ("shoot-through")
        GND
```

The L293D contains **four half-H drivers**; channels 1 and 2 form one full H-bridge. Each half-H is driven by a single input (IN1 or IN2) that switches its upper and lower transistors in a complementary way, so a single input cannot short its own leg.

Pin	Function
1 EN1,2	Enables channels 1 and 2 (our PWM input)
2 IN1, 7 IN2	Direction inputs
3 OUT1, 6 OUT2	To the motor
4, 5, 12, 13	Ground (and heat sink)
8 VS (VCC2)	Motor supply, up to 36 V (12 V here)
16 VSS (VCC1)	Logic supply, 4.5–7 V (5 V here)

EN1	IN1	IN2	OUT1	OUT2	Motor
1	1	0	High	Low	**Forward**
1	0	1	Low	High	**Reverse**
1	0	0	Low	Low	Brake (terminals shorted through the low-side transistors)
1	1	1	High	High	Brake
0	x	x	Off	Off	**Coast** (free-wheeling)

Key characteristics: about 600 mA continuous per channel (1.2 A peak); TTL-compatible inputs (a 3.3 V logic high is accepted); internal **clamp (flyback) diodes** — the “D” in L293D; thermal shutdown. It uses bipolar output transistors, so it drops a few volts at high current; modern MOSFET drivers (TB6612FNG, DRV8833) are more efficient.

Flyback diodes: a motor winding is an inductor. When a transistor switches off, the current cannot stop instantly: V = L·di/dt produces a large voltage spike. The diodes give that current a path back to the supply rail, clamping the spike and protecting the transistors.

Applying PWM to EN rather than to IN: when EN goes low, both outputs switch off and the motor **coasts** during the off-time. The direction pins stay constant, which keeps the logic simple.

6.8 Direction control, ramps and safe reversal

```
        static void motor_set_direction(unsigned int dir)
        {
            if (dir == DIR_FORWARD) {
                IO0CLR = IN2_PIN;
                IO0SET = IN1_PIN;
            } else if (dir == DIR_REVERSE) {
                IO0CLR = IN1_PIN;
```

```
            IO0SET = IN2_PIN;
        } else {
            IO0CLR = IN1_PIN | IN2_PIN;
        }
        g_direction = dir;
    }
```

The pin that must go low is cleared *before* the other is set, so IN1 and IN2 are never both high, even for a moment.

```
    static void motor_ramp(unsigned int target, unsigned int ramp_ms)
    {
        unsigned int now = g_speed;
        unsigned int diff = (target > now) ? (target - now) : (now - target);
        unsigned int step_ms;

        if (diff == 0U) {
            return;
        }

        step_ms = udiv(ramp_ms, diff);

        while (now != target) {
            if (target > now) {
                now++;
            } else {
                now--;
            }
            motor_set_speed(now);
            delay_ms(step_ms);
        }
    }
```

- Step time = ramp_ms / |Δduty| using integer division (`udiv`).
- Forward: 500 / 75 = 6 ms → 75 × 6 = **450 ms**. Reverse: 500 / 40 = 12 ms → 40 × 12 = **480 ms**.
- Ramp-downs: 300 / 75 = 4 ms → **300 ms**; 300 / 40 = 7 ms → **280 ms**.

```
    /* Ramp down and pause before any change of direction */
    static void motor_run_step(const motor_step_t *s)
    {
        unsigned int target = (s->direction == DIR_STOP) ? 0U : s->speed;

        if ((s->direction != g_direction) && (g_speed > 0U)) {
            motor_ramp(0U, s->ramp_ms);
            motor_set_direction(DIR_STOP);
            delay_ms(DIR_CHANGE_PAUSE_MS);
        }

        if (s->direction != DIR_STOP) {
            motor_set_direction(s->direction);
            motor_ramp(target, s->ramp_ms);
        } else {
            motor_set_direction(DIR_STOP);
            motor_set_speed(0U);
        }

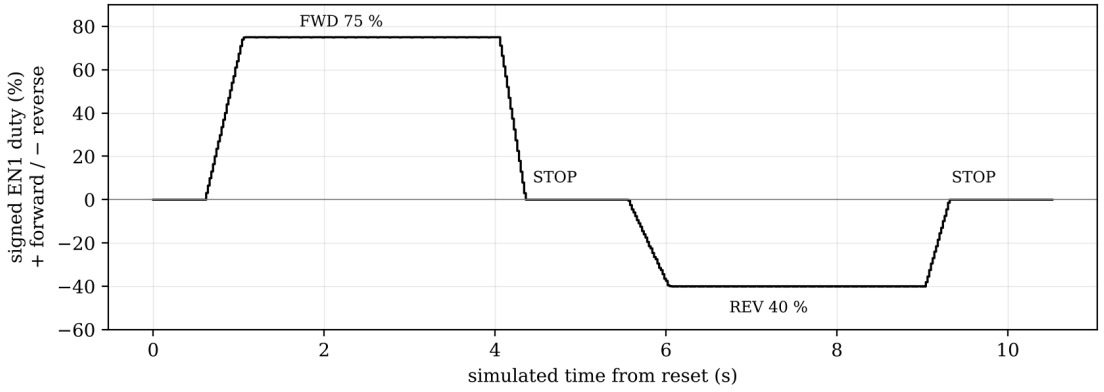
        delay_ms(s->run_ms);
    }
```

Reversal sequence: ramp to 0 % → IN1 = IN2 = 0 → wait `DIR_CHANGE_PAUSE_MS` (200 ms) for the motor to slow → set the new direction → ramp up. In emulation the direction never changed directly between forward and reverse, and IN1 and IN2 were never high together.

6.9 Profile cycle timing

Step	Ramp	Pause	Run	Subtotal
0 Forward 75 %	450 ms	—	3000 ms	3450 ms
1 Stop	300 ms (75 → 0)	200 ms	1000 ms	1500 ms
2 Reverse 40 %	480 ms	—	3000 ms	3480 ms
3 Stop	280 ms (40 → 0)	200 ms	1000 ms	1480 ms
Cycle				**9910 ms ≈ 9.9 s**

The emulator measurement agrees: forward drive began at about 0.6 s and the first cycle ended at about 10.5 s.



6.10 Integer division without hardware — `udiv()`

```c
/* ARM7TDMI has no divide instruction */
static unsigned int udiv(unsigned int n, unsigned int d)
{
    unsigned int q = 0U;
    unsigned int bit = 1U;

    if (d == 0U) {
        return 0U;
    }
    while ((d < n) && ((d & 0x80000000U) == 0U)) {
        d <<= 1;
        bit <<= 1;
    }
    while (bit != 0U) {
        if (n >= d) {
            n -= d;
            q |= bit;
        }
        d >>= 1;
        bit >>= 1;
    }
```

```
            return q;
        }
```

This is **restoring binary long division**: shift the divisor left until it exceeds the dividend, then subtract it back down bit by bit, setting a quotient bit each time a subtraction succeeds. Cost ≈ 2 × 32 loop iterations at worst. It exists because the ARM7TDMI has no divide instruction and `ld` is called without `libgcc` (which provides `__aeabi_uidiv`). Divisions by **constants**, like `PCLK_HZ / 1000U`, are computed by the compiler at build time and need no routine.

PART 7 — TOOLCHAIN, START-UP CODE, LINKER SCRIPT

7.1 From source to flash: the build pipeline

Stage	Tool	Input → Output	What happens
Preprocess	`gcc` (cpp)	`main.c` → expanded C	`#define`s replaced, `#if USE_HW_PWM` resolved
Compile	`gcc` (cc1)	C → assembly	C turned into ARM instructions
Assemble	`as`	assembly → `main.o`, `startup.o`	Machine code + symbol table + relocations
Link	`ld -T linker.ld`	`.o` files → `firmware.elf`	Sections placed at final addresses; symbols resolved
Convert	`objcopy -O ihex`	ELF → `firmware.hex`	Raw program bytes in Intel HEX text format

The four commands:

```
arm-none-eabi-as  -mcpu=arm7tdmi -g -o startup.o startup.s
arm-none-eabi-gcc -c -mcpu=arm7tdmi -marm -O0 -g -Wall -Wextra -ffreestanding -fno-builtin
main.c -o main.o
arm-none-eabi-ld  -T linker.ld -o firmware.elf startup.o main.o
arm-none-eabi-objcopy -O ihex firmware.elf firmware.hex
```

What “arm-none-eabi” means: target **ARM**, **no** operating system, **EABI** (Embedded Application Binary Interface — the conventions for calling functions, passing arguments in R0–R3, stack alignment, and object-file format).

Flag	Purpose
`-mcpu=arm7tdmi`	Generate only instructions this core supports (ARMv4T)
`-marm`	Use 32-bit ARM instructions, not 16-bit Thumb
`-O0`	No optimisation: code follows the source exactly; easiest to debug and step through
`-g`	Emit DWARF debug information; Proteus uses it to show variables by name
`-Wall -Wextra`	Enable warnings; the project builds with **zero** warnings
`-ffreestanding`	No hosted environment: no assumptions about `main`'s arguments or a C library
`-fno-builtin`	Don't replace code with built-in library calls (e.g. a loop with `memset`)
`-c`	Compile only, don't link
`-T linker.ld`	Use our linker script instead of the default

Why call `ld` directly instead of linking through `gcc`? It guarantees that nothing extra is linked: no C start-up files (`crt0`), no C library, no `libgcc`. Every byte in the image comes from our two source files. The trade-off is that helper functions such as division are not available — hence `udiv()`.

7.2 ELF, HEX and the symbol table

ELF (Executable and Linkable Format) contains section headers, the program bytes, the **symbol table** (names and addresses of every function and variable) and debug information. Proteus can load it directly and display `g_fail_code` and other variables by name.

Real symbol map of the final firmware (`arm-none-eabi-nm -n firmware.elf`, excerpt):

Address	Type	Symbol
0x0000 0000	T	`_start` (vector table)
0x0000 0020	T	`Reset_Handler`
0x0000 0404	t	`spi_transfer`
0x0000 0640	t	`ee_write_page`
0x0000 0EF0	t	`profile_load`
0x0000 1324	T	`main`
0x0000 1530	t	`default_steps` (constant table in flash)
0x4000 0000	b	`steps` (RAM)
0x4000 003C	B	`g_fail_code`
0x4000 0040	B	`g_profile_source`
0x4000 0048	B	`g_step`

Symbol types: **T/t** = code (text), **B/b** = zero-initialised data (bss), **D** = initialised data, **A** = absolute value; uppercase = global, lowercase = local (`static`).

Section sizes: `.vectors` 0x20 bytes, `.text` 0x1528 bytes (code and constants), `.data` 0, `.bss` 0x58 bytes → `size` reports **text 5448, data 0, bss 88**.

Intel HEX format: each line is `:LLAAAATT<data>CC` — LL = number of data bytes, AAAA = address, TT = record type (00 data, 01 end of file, 04 extended address), CC = checksum.

Our first record: `:10000000060000EA170000EA170000EA170000EAFD` - `10` = 16 bytes, `0000` = address 0, `00` = data, then the first four vectors (`EA000006` stored little-endian as `06 00 00 EA`, ...), and `FD` = checksum. - The HEX checksum uses **the same two's-complement idea as our profile checksum**: the sum of all bytes in the record including CC is 0 mod 256.

The last record, `:00000001FF`, marks end of file.

7.3 Start-up code — line by line

```
.syntax unified
.cpu arm7tdmi
.arm

.section .vectors, "ax", %progbits
.align 2
.global _start

_start:
    b Reset_Handler
    b Undefined_Handler
    b SWI_Handler
    b PrefetchAbort_Handler
    b DataAbort_Handler
    b Reserved_Handler
    b IRQ_Handler
    b FIQ_Handler

.section .text.startup, "ax", %progbits
.align 2
.global Reset_Handler
.extern main

Reset_Handler:
    ldr sp, =_stack_top

    ldr r0, =_sidata
    ldr r1, =_sdata
    ldr r2, =_edata

copy_data:
    cmp r1, r2
    bhs clear_bss_start
    ldrb r3, [r0], #1
    strb r3, [r1], #1
    b copy_data

clear_bss_start:
    ldr r1, =_sbss
    ldr r2, =_ebss
    mov r3, #0
```

```
clear_bss:
    cmp r1, r2
    bhs call_main
    strb r3, [r1], #1
    b clear_bss

call_main:
    bl main

stop:
    b stop

Undefined_Handler:
    b Undefined_Handler

SWI_Handler:
    b SWI_Handler

PrefetchAbort_Handler:
    b PrefetchAbort_Handler

DataAbort_Handler:
    b DataAbort_Handler

Reserved_Handler:
    b Reserved_Handler

IRQ_Handler:
    b IRQ_Handler

FIQ_Handler:
    b FIQ_Handler
```

Part	Explanation
`.syntax unified`, `.cpu arm7tdmi`, `.arm`	Assembler settings: unified syntax, target core, ARM (32-bit) instructions
`.section .vectors, "ax"`	Puts the vector table in its own allocatable, executable section so the linker can place it at address 0
`_start: b Reset_Handler …`	The eight exception vectors
`ldr sp, =_stack_top`	Loads the stack pointer with the top of RAM (0x4000 4000); the stack grows downward. The `=` form puts the constant in a nearby literal pool and loads it PC-relative
`copy_data` loop	Copies initial values of global variables from flash (`_sidata`) to RAM (`_sdata` … `_edata`), one byte at a time with post-indexed `ldrb` / `strb`
`clear_bss` loop	Writes zero to every byte from `_sbss` to `_ebss`; C requires uninitialised globals to start at 0
`bl main`	Calls `main`; `BL` stores the return address in LR
`stop: b stop`	Safety net if `main` ever returns (it never does)
Handler stubs	Each exception handler is an infinite loop; with interrupts disabled none should occur, and if one does, the CPU parks in a known place where a debugger can see it

Why set the stack first? C functions push return addresses and local variables onto the stack; without a valid SP the first function call would write to an undefined address.

Why `.data` must be copied: a variable such as `int x = 5;` must be in RAM to be writable, but RAM is empty at power-up. The initial value is stored in flash, and start-up copies it across. This firmware happens to have no initialised variables (`.data` is empty), but the start-up code is general.

Byte-wise copy: simple and safe for any alignment; for large sections a word-wise copy (or `LDM` / `STM`) would be faster.

7.4 Linker script — line by line

```
ENTRY(_start)

MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 128K
    RAM   (rwx) : ORIGIN = 0x40000000, LENGTH = 16K
}

_stack_top = ORIGIN(RAM) + LENGTH(RAM);

SECTIONS
{
    .vectors :
    {
        KEEP(*(.vectors))
```

```
    } > FLASH

    .text :
    {
        *(.text.startup)
        *(.text*)
        *(.rodata*)
        . = ALIGN(4);
    } > FLASH

    .data :
    {
        . = ALIGN(4);
        _sdata = .;
        *(.data*)
        . = ALIGN(4);
        _edata = .;
    } > RAM AT > FLASH

    _sidata = LOADADDR(.data);

    .bss (NOLOAD) :
    {
        . = ALIGN(4);
        _sbss = .;
        *(.bss*)
        *(COMMON)
        . = ALIGN(4);
        _ebss = .;
    } > RAM
}
```

Statement	Meaning
`ENTRY(_start)`	Entry symbol recorded in the ELF header (entry point 0x0)
`MEMORY { FLASH …; RAM … }`	Physical memory regions: FLASH at 0x0 (128 KB used of 512 KB), RAM at 0x4000 0000 (16 KB used of 32 KB)
`_stack_top = ORIGIN(RAM) + LENGTH(RAM)`	0x4000 4000, the initial SP
`.vectors : { KEEP(*(.vectors)) } > FLASH`	Vectors first, at address 0; `KEEP` stops the linker discarding them
`.text : { … } > FLASH`	Start-up code, program code (`.text*`) and constants (`.rodata*`)
`.data : { … } > RAM AT > FLASH`	Runs in RAM (**VMA**) but is stored in flash (**LMA**)
`_sidata = LOADADDR(.data)`	Flash address of the initial values, used by `copy_data`
`.bss (NOLOAD) : { … } > RAM`	Zero-initialised variables; NOLOAD → takes no space in the image
`_sdata`, `_edata`, `_sbss`, `_ebss`	Boundary symbols exported to the start-up code

VMA vs LMA: the **Virtual Memory Address** is where a section lives while the program runs; the **Load Memory Address** is where it is stored in the image. For code they are the same (flash). For `.data` they differ (run in RAM, stored in flash) — that difference is exactly what the copy loop bridges.

RAM usage: 88 bytes of `.bss` at the bottom of RAM, the stack at the top growing down; there are almost 16 KB between them. The largest stack user is `profile_load()`, with a 53-byte buffer.

7.5 Why `volatile` matters — with an example

Without `volatile`, the compiler may assume memory only changes when the program writes it:

```
        while ((S0SPSR & 0x80) == 0) { }   /* without volatile, could become an endless loop */
```

The compiler could read S0SPSR once, keep it in a register and loop forever, because it sees nothing in the loop that changes it. With `volatile`, every iteration performs a real load from the hardware register. The same applies to: - every peripheral register (through the `REG32` macro), - the delay loop counter in `short_delay()` (otherwise the loop could be deleted), - the `g_*` debug variables (so they are always really written to RAM, where the debugger reads them).

PART 8 — FIRMWARE WALKTHROUGH

8.1 Configuration section

The top of `main.c` defines, in order: the `USE_HW_PWM` option; register addresses via `REG32`; clock constants (`CCLK_HZ` = 12 MHz, `PCLK_HZ` = CCLK/4); pin masks (`LED_PIN`, `CS_PIN`, `EN1_PIN`, `IN1_PIN`, `IN2_PIN`); PINSEL values; peripheral bit names; EEPROM opcodes and status bits; time-out limits; error codes; direction codes; profile layout constants; and the default profile table.

Why `#define` constants instead of magic numbers? Self-documenting code, one place to change a value, and the compiler folds expressions such as `(PCLK_HZ / 1000U) - 1U` into a single constant at build time.

Why `static`? Functions and module variables are `static`, so they are private to this file; only `main` and the `g_*` debug variables are global. This prevents name clashes and lets the compiler know the full use of each function.

Why the `U` suffix (e.g. `1U << 31`)? It makes the constant unsigned. Shifting a signed 1 into the sign bit is undefined behaviour in C.

8.2 `main()` — the application sequence

```
int main(void)
{
    unsigned int i;

    PINSEL0 = (PINSEL0 & ~PINSEL0_MASK) | PINSEL0_VALUE;

    IO0SET = CS_PIN;
    IO0CLR = LED_PIN | EN1_PIN | IN1_PIN | IN2_PIN;
    IO0DIR |= LED_PIN | CS_PIN | EN1_PIN | IN1_PIN | IN2_PIN;

    VPBDIV = 0U;
    timers_init();
    pwm_init();
    motor_set_direction(DIR_STOP);

    for (i = 0; i < 3U; i++) {
        IO0SET = LED_PIN;
        delay_ms(100U);
        IO0CLR = LED_PIN;
        delay_ms(100U);
    }

    PCONP |= PCONP_PCSPI0;
    S0SPCCR = 8U;
    S0SPCR = SPCR_MSTR;

    g_fail_code = profile_load();
    g_step_count = step_count;

    /* EEPROM error: keep the motor off and blink the LED */
    if (g_fail_code != FAIL_NONE) {

        while (1) {
            IO0SET = LED_PIN;
            delay_ms(100U);
            IO0CLR = LED_PIN;
```

```
                delay_ms(100U);
            }
        }

        IO0SET = LED_PIN;

        while (1) {
            for (i = 0; i < step_count; i++) {
                g_step = i;
                motor_run_step(&steps[i]);
            }
            g_cycles++;
        }
    }
```

Phase	Lines	Notes
Pin set-up	PINSEL0, IO0SET, IO0CLR, IO0DIR	CS is set **high before** it becomes an output; LED and motor pins start low → motor off from the first instruction
Clocks	`VPBDIV = 0`	PCLK = CCLK/4, set explicitly
Timers and PWM	`timers_init`, `pwm_init`, `motor_set_direction(DIR_STOP)`	Motor guaranteed stopped before anything else runs
Start-up signature	3 blinks of 100 ms	Shows that *this* firmware is running (this resolved an early confusion with a GPIO-only test program)
SPI	PCONP, `S0SPCCR = 8`, `S0SPCR = MSTR`	Master, mode 0, 375 kHz
Profile	`g_fail_code = profile_load()`	All EEPROM work happens here
Error policy	blink forever at 5 Hz	**Fail safe:** the motor is never enabled without a valid profile
Run	LED on; loop over `steps[]` forever; count `g_cycles`	`g_step` shows progress in the Watch Window

8.3 Error codes and debug variables

`g_fail_code`	Name	Detected in	Typical cause
0	FAIL_NONE	—	—
1	FAIL_SPI_TIMEOUT	`spi_transfer`	SPI not clocking, wrong pin-mux
2	FAIL_SPI_FAULT	`spi_transfer`	MODF (SSEL active) or ABRT
3	FAIL_NO_RESPONSE	`ee_check`	Status = 0xFF: EEPROM unpowered, SO open, CS not connected
4	FAIL_PROTECTED	`ee_check`	BP1:BP0 = 11
5	FAIL_WEL_NOT_SET	`ee_write_page`	SI open or SO stuck low
6	FAIL_WRITE_TIMEOUT	`ee_wait_ready`	WIP never clears
9	FAIL_PROFILE_BAD	`profile_load`	Defaults written but read back invalid

Variable	Meaning
`g_profile_source`	1 = valid profile found; 2 = defaults written and verified
`g_step_count`	Number of steps loaded
`g_step`	Step currently running (0-based)
`g_direction`	0 stop, 1 forward, 2 reverse
`g_speed`	Current duty in %
`g_cycles`	Number of completed profile cycles

8.4 Test programs

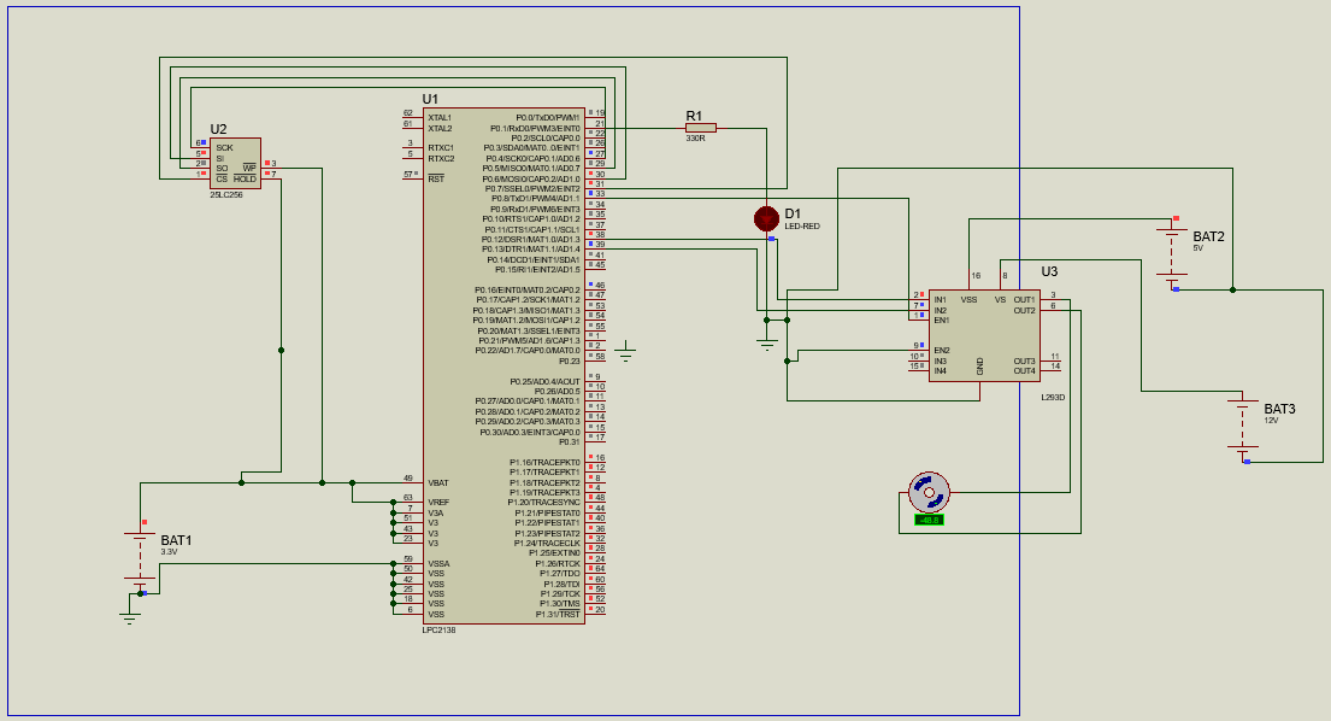
Program	Purpose	Pass criterion
`tests/spi_eeprom_test.c`	Isolates SPI and EEPROM: status read, WREN/WEL check, writes 0x5A then 0xA5 at 0x0010 and reads each back	3 blinks, then steady LED
`tests/motor_test.c`	Isolates the L293D and motor: EN1 held high, direction toggled; no EEPROM, no timers, no PWM	Motor turns forward, stops, reverses

Why two different test values? If only 0x5A were written and the write silently failed, a leftover 0x5A from an earlier run would still read back correctly — a false pass. Writing 0x5A and then its bitwise complement 0xA5 (`01011010` → `10100101`) makes every bit change, so a failed write cannot pass.

PART 9 — SIMULATION, EMULATION AND VERIFICATION

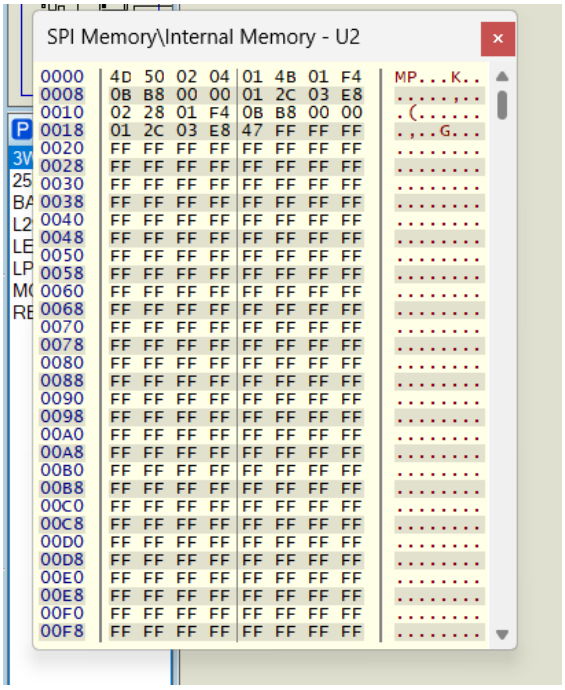
9.1 How Proteus VSM works

- **Mixed-mode simulation:** an analog SPICE-based solver (batteries, resistors, L293D outputs, motor) coupled to an event-driven digital simulator (logic levels, pin states).
- **Microcontroller models** execute the real compiled program instruction by instruction and model the on-chip peripherals; loading the ELF gives source-level debugging and named variables.
- **Interactive parts:** the LED lights, the motor rotor animates and shows RPM, and pin indicators show logic states (red = high, blue = low, grey = floating).
- **CPU load** in the status bar shows whether the simulation keeps up with real time: above 100 % means it is running slower than real time.
- **Persistent model data:** EEPROM contents are kept between runs (and can be cleared with *Debug → Reset Persistent Model Data*), which mimics a real power cycle.



9.2 Proteus results

Test	Observation	Conclusion
GPIO test	LED constantly on	Toolchain, start-up code and loading work
EEPROM self-test	3 blinks, then steady LED	SPI wiring, chip select, both data directions and EEPROM supply verified
Final firmware	Motor rotating; IN1 high, IN2 low (forward step); 00:03:36 simulated at 29 % CPU	Profile loaded and executed in real time
EEPROM memory window	`4D 50 02 04 01 4B 01 F4 0B B8 … 47`, rest `FF`	Stored profile matches the design byte for byte, checksum correct



9.3 Host-side emulation — what it is and why it was used

What: the compiled `firmware.bin` runs instruction by instruction on a PC in an ARM CPU emulator (the Unicorn engine). Accesses to peripheral addresses are intercepted by small Python models: an SPI0 model, a 25LC256 model (WREN, RDSR, READ, WRITE, WIP timing, page wrap), GPIO, Timer0/1 and the PWM unit.

Why: some conditions are hard to create in Proteus — a corrupt EEPROM, a stuck MISO line, an SPI that never clocks. In the emulator they are one-line changes, and every pin edge can be measured precisely.

Results:

Injected condition	Expected	Result
Blank EEPROM	Write 29-byte default, run	One page write at 0x0000; profile ran ✓
Valid stored profile	Run it, no write	No writes; stored steps executed ✓
Corrupted byte	Checksum fails → rewrite defaults	✓
Old version	Rejected → rewrite	✓
EEPROM not answering	Error 3, motor off	✓
MISO stuck low	Error 5, motor off	✓
SPI not clocking	Error 1, motor off	✓

The measured EN1 duty was exactly 75 % forward and 40 % reverse, with a cycle of about 9.9 s. IN1 and IN2 were never high together, and the direction never switched while the motor was powered.

Limitations of emulation: the models are simplified (not cycle-accurate, ideal timing), so emulation tests the firmware's *logic*; it does not replace Proteus or hardware.

9.4 Verification levels

Level	Evidence	Status
Static	Builds with zero warnings at `-Wall -Wextra`; no unresolved symbols	✓
Unit / logic	Emulator fault injection	✓
System simulation	Proteus: EEPROM test, motor run, memory dump	✓
Hardware	Real board	Not yet done

PART 10 — DEBUGGING CASE STUDIES

Interviewers value *how you solved problems* more than the final result. Tell each story in **STAR** form: **S**ituation, **T**ask, **A**ction, **R**esult.

Case 1 — Simulation would not start (power supply)

- **Situation:** Proteus reported “VBAT (0.00 V)” and “3.3V/VDD (5.00 V)”, and later “singular matrix … #BAT1#branch, GMIN stepping failed”.
- **Analysis:** the LPC2138’s VDD pin sat on the default 5 V power net, and VBAT was unconnected. A singular matrix means the analog solver could not find a consistent solution — typically an ideal voltage source in an impossible loop or a floating node.
- **Action:** every LPC2138 supply pin (V3, V3A, VREF and VBAT) was tied to a single 3.3 V source, grounds were joined, and the battery wiring was rearranged.
- **Result:** the simulation started. **Lesson:** check supplies first; a simulator error message names the faulty node.

Case 2 — EEPROM test was inconclusive

- **Situation:** the first SPI test sometimes left the LED off and sometimes turned it on after a delay; nobody could say whether it had passed.
- **Analysis (four separate flaws):**
  1. No time-out: if SPIF never set, the CPU hung with the LED off — identical to a failure.
  2. A fixed delay instead of WIP polling.
  3. A single test value (0x5A): stale data from an earlier run could give a false pass.
  4. The final LED state was the same as a previous GPIO test program, and that program had once been loaded by mistake.
- **Action:** rewrote the test with time-outs on every wait, WIP polling, a WEL check after WREN, two complementary values (0x5A, 0xA5), a 3-blink start-up signature and numbered fail codes.
- **Result:** “3 quick blinks, then steady LED” — an unambiguous pass. **Lesson:** a test must be able to fail visibly.

Case 3 — Chip select had no effect

- **Situation:** the original code wrote `PINSEL0 = 0x5500`.
- **Analysis:** bits 15:14 = 01 select SSEL0 on P0.7, so GPIO writes to P0.7 were ignored, and in master mode SSEL0 is an input that can trigger a mode fault.
- **Action:** `PINSEL0 = 0x1500` — SPI0 on P0.4–P0.6, P0.7 as GPIO.
- **Result:** software chip select works. **Lesson:** verify the pin-function table bit by bit against the user manual.

Case 4 — Build failed: `linker.ld:1: syntax error`

- **Situation:** the assembler worked, but the linker failed on line 1.
- **Analysis:** the linker script file contained the wrong text (the assembly start-up code had been pasted into it). Reproducing the error by feeding assembly text to `ld` gave exactly the

same message.

- **Action:** recreated `linker.ld` and checked its first line with `type linker.ld`.
- **Result:** clean build. **Lesson:** when a tool fails on line 1, look at the file itself before the logic.

Case 5 — “The motor does not turn”

- **Situation:** with the first motor firmware, the motor showed 0.00 RPM.
- **Analysis:** the pin indicators showed IN1 high, and IN1 is only driven *after* a successful profile load, so the firmware was working. The status bar showed less than 1 s of simulated time with CPU load above 100 %: the 3 kHz PWM gave the analog solver 6000 edges per second, so the simulation crawled, and the ramp had barely started.
- **Action:** replaced the 3 kHz hardware PWM with 100 Hz timer-based software PWM (≈ 30× fewer edges), shortened the start-up and ramps, and wrote `motor_test.c` to separate driver faults from firmware faults.
- **Result:** CPU load 29 %, real-time simulation, motor running the profile. **Lesson:** read what the tools show (pin states, status bar) before changing code.

PART 11 — DESIGN ALTERNATIVES AND TRADE-OFFS

Decision	Alternatives	Why this choice
External SPI EEPROM	Internal flash via IAP; I²C EEPROM (24LC256); FRAM	Byte-writable and simple; flash needs sector erase and IAP calls; I²C needs addressing, ACKs and pull-ups; FRAM is faster and has near-unlimited endurance but costs more
SPI0 peripheral	SSP (the second SPI controller, with an 8-frame FIFO); bit-banging	SPI0 is simple and sufficient; SSP would allow faster clocks and FIFO transfers; bit-banging wastes CPU
GPIO chip select	Hardware SSEL0	SSEL0 is an input in master mode (mode fault); multi-byte commands need CS held low
Polling	Interrupts (VIC)	Single-task, deterministic, easy to verify; time-outs make it safe. Interrupts would free the CPU for other work
Additive checksum	CRC-8/16/32; Hamming ECC	Simple and transparent; a CRC is stronger (detects swaps and bursts)
Version field + magic	No header	Safe firmware upgrades; blank EEPROM detection
Software PWM at 100 Hz	Hardware PWM4 (kept as an option); higher frequencies	Real-time simulation, and independent of the PWM-peripheral model; hardware PWM at ≥ 20 kHz is better on real boards
L293D	L298N; MOSFET drivers (TB6612FNG, DRV8833); discrete MOSFET bridge	Built-in diodes, logic-level inputs, a Proteus model, adequate current; MOSFET drivers waste less voltage
12 MHz, no PLL	60 MHz with PLL and MAM	Simpler, exact timing maths, ample performance for this load
Bare-metal, no HAL	NXP sample libraries; an RTOS	Full understanding and control; tiny image (5.4 KB); no hidden behaviour
`-O0`	`-O2` / `-Os`	Code matches the source exactly, for teaching and debugging; release builds would optimise

LPC2138 vs LPC2148: similar ARM7TDMI-S parts and pin-outs; the LPC2148 adds USB 2.0 and 40 KB RAM. The firmware would carry over but needs re-testing.

PART 12 — LIMITATIONS AND PRODUCTION ROADMAP

Area	Current state	Production improvement
Validation	Simulation and emulation only	Hardware bring-up; logic-analyser capture of SPI; oscilloscope on PWM; measured RPM vs duty
Speed control	Open loop	Encoder or Hall sensor + PI controller (closed-loop RPM)
PWM	Software, 100 Hz, only while waiting	Hardware PWM at 20 kHz, or timer-interrupt PWM
Integrity	8-bit additive checksum	CRC-16
Power-loss safety	One profile copy	A/B copies with sequence numbers; verify before switching
Flexibility	Profile edited in code	UART command interface; several stored profiles selectable by button
Concurrency	Polling only	Interrupt-driven SPI and timers; low-power idle between events
Clocking	12 MHz, no PLL	PLL to 60 MHz, MAM configured
Safety	Ramps and dead time	Over-current sensing, stall detection, watchdog timer
Boot	Vector checksum inserted by the ISP tool	Post-build step that writes the checksum into the image

PART 13 — FORMULA AND NUMBERS SHEET

Quantity	Formula	Value
CPU clock	Crystal, no PLL	CCLK = 12 MHz
Peripheral clock	CCLK / 4 (VPBDIV = 0)	PCLK = 3 MHz
SPI clock	PCLK / S0SPCCR	3 MHz / 8 = **375 kHz**
SPI byte time	8 / f_SCK	**21.3 µs**
Profile read time	(3 + 53) × 21.3 µs	≈ 1.2 ms
Timer1 tick	(PR + 1)/PCLK, PR = 2999	**1 ms**
Timer0 tick	PR = 299	**100 µs**
PWM period	(MR0 + 1) × 100 µs, MR0 = 99	**10 ms → 100 Hz**
PWM resolution	1 / 100	**1 %**
Average voltage	D × V_S	9.0 V (75 %), 4.8 V (40 %)
Forward ramp	⌊500/75⌋ × 75	6 ms × 75 = **450 ms**
Reverse ramp	⌊500/40⌋ × 40	12 ms × 40 = **480 ms**
Cycle time	3450 + 1500 + 3480 + 1480	**≈ 9.9 s**
Profile size	4 + 6N + 1	29 bytes (N = 4), 53 max
Checksum	(−Σ bytes) mod 256	**0x47**
LED current	(3.3 − V_F)/330	≈ 3.9 mA
PLL for 60 MHz	M = 5, P = 2	F_CCO = 240 MHz
32-bit ms counter wrap	2^{32} ms	≈ 49.7 days
Firmware size	text / data / bss	5448 / 0 / 88 bytes
Stack top	ORIGIN(RAM) + 16 KB	0x4000 4000
Boot checksum word needed at 0x14	−Σ(other 7 vectors)	0x99FF FF70

Register values:

Register	Value	Meaning
PINSEL0	0x0000 1500	SPI0 on P0.4–P0.6
S0SPCR	0x20	Master, mode 0, MSB first
S0SPCCR	8	SCK = PCLK/8
T1PR / T0PR	2999 / 299	1 ms / 100 µs ticks
T0MR0 / T0MCR	99 / 0x2	Reset TC at 99
VPBDIV	0	PCLK = CCLK/4
EEPROM opcodes	WREN 06 · RDSR 05 · WRITE 02 · READ 03	—

PART 14 — 100 INTERVIEW AND VIVA QUESTIONS WITH ANSWERS

A. Project overview (1–12)

1. Explain your project in one minute. See the answer in Section 1.2: profile stored in a 25LC256 over SPI, validated at boot (magic, version, range, checksum), defaults restored and verified if invalid, executed on a DC motor through an L293D with PWM, ramps and safe reversal; bare-metal, verified in Proteus.

2. What problem does it solve? It lets a machine's motion sequence be stored as data in non-volatile memory, so it survives power loss and can be changed without rewriting the program logic.

3. What is a "motor profile"? An ordered list of steps; each step has a direction, a speed (PWM %), a ramp time and a run time. The controller executes the steps in sequence and repeats them.

4. What are the main hardware blocks? LPC2138 controller, 25LC256 EEPROM on SPI0, L293D H-bridge driving a DC motor, and a status LED, with 3.3 V, 5 V and 12 V supplies sharing a common ground.

5. Which parts did you write yourself? The start-up assembly, linker script, SPI driver, EEPROM driver, profile format and validation, PWM, ramp and reversal logic, and the test programs — all at register level, with no vendor libraries. *(Adapt this to your own contribution.)*

6. What happens at power-up? Start-up code initialises the stack and RAM; `main` sets up pins and timers, blinks the LED three times, starts SPI, loads or restores the profile, then turns the LED on and runs the profile forever. Any error stops the motor and blinks the LED.

7. What happens if the EEPROM is blank? It reads 0xFF, so the magic check fails; the firmware writes the default profile, reads it back, validates it and runs it (`g_profile_source = 2`).

8. What if the EEPROM is removed? The status read returns 0xFF, `ee_check` reports error 3, the motor stays off and the LED blinks at 5 Hz.

9. How do you change the profile? Edit `default_steps[]` and increment `PROFILE_VERSION`; on the next boot the old stored profile is rejected because of the version mismatch, and the new one is written.

10. What was the most challenging part? Debugging with limited visibility — for example the motor "not turning", which turned out to be the simulator running far slower than real time because of a 3 kHz PWM (Case 5).

11. How did you verify the design? Clean build with all warnings enabled; emulator fault injection; Proteus tests (EEPROM self-test, motor run, EEPROM memory dump). Hardware testing is the next step.

12. What would you do differently? Hardware PWM at 20 kHz on real hardware, CRC-16, A/B profile copies, interrupts, closed-loop speed control, and a UART interface.

B. ARM7 and LPC2138 (13–32)

13. What does ARM7TDMI-S stand for? ARM7 core with Thumb, Debug, enhanced Multiplier, EmbeddedICE; “-S” means synthesizable.

14. RISC or CISC? Harvard or von Neumann? RISC, load/store; von Neumann (a single bus for instructions and data).

15. Describe the pipeline. Three stages — fetch, decode, execute. Because of it, the PC reads as the current instruction + 8 in ARM state.

16. How many registers does ARM7 have? 37: 31 general-purpose (16 visible at a time, R0–R15) and 6 status registers (CPSR plus 5 SPSRs).

17. What are R13, R14, R15? Stack pointer, link register (return address), program counter.

18. What are the processor modes? User, FIQ, IRQ, Supervisor, Abort, Undefined, System. Exception modes have banked SP and LR and their own SPSR; FIQ also banks R8–R12.

19. Which mode is the core in after reset? Supervisor mode, ARM state, IRQ and FIQ disabled.

20. What is in the CPSR? Condition flags N, Z, C, V; interrupt-disable bits I and F; the Thumb bit T; and the mode bits M[4:0].

21. ARM vs Thumb state? ARM uses 32-bit instructions (more powerful, faster per instruction); Thumb uses 16-bit instructions (about 30 % smaller code). We compile with `-marm`.

22. Why is there a function called `udiv`? The ARM7TDMI has no divide instruction and we don’t link libgcc, so division is implemented in software by shift-and-subtract.

23. What is the vector table? Eight instructions at address 0, one per exception (reset, undefined, SWI, prefetch abort, data abort, reserved, IRQ, FIQ), normally branches to handlers.

24. What is special about address 0x14 on the LPC2000? The boot loader treats user code as valid only if the 8 vector words sum to zero; the reserved word at 0x14 holds the balancing value. For our image it would be 0x99FFFF70; ISP tools insert it.

25. How is code programmed into an LPC2138? Through ISP over UART0 (hold P0.14 low at reset; for example Flash Magic) or through JTAG.

26. Explain CCLK and PCLK. CCLK is the CPU clock (12 MHz here, crystal without PLL); PCLK is the peripheral clock = CCLK / VPBDIV divisor — /4 after reset → 3 MHz.

27. How would you run the CPU at 60 MHz? With the PLL: M = 5, P = 2 (F_CCO = 240 MHz, inside the 156–320 MHz range); write PLLCFG and PLLCON, feed 0xAA/0x55, wait for lock, connect, feed again; then configure the MAM for flash wait states.

28. What is the MAM? The Memory Accelerator Module: it prefetches and buffers flash lines so the core can run faster than the flash access time allows.

29. What is the VIC? The Vectored Interrupt Controller: it prioritises up to 16 vectored IRQ sources plus non-vectored ones and supplies the handler address. It is not used here because the design is polled.

30. How do you select pin functions? With PINSEL0/1; two bits per pin choose one of up to four functions. PINSEL0 = 0x1500 selects SCK0, MISO0 and MOSI0.

31. Why use IO0SET/IO0CLR rather than IO0PIN? They change only the pins written as 1 in a single store — atomic and faster than read-modify-write.

32. What is PCONP for? It enables or disables the clock to each peripheral to save power; bits for Timer0, Timer1, PWM and SPI0 are set.

C. SPI (33–46)

33. What is SPI? A synchronous, full-duplex, master–slave serial bus with SCK, MOSI, MISO and one chip select per slave.

34. Why is SPI called full-duplex? On each clock edge the master sends one bit on MOSI while receiving one on MISO; the two shift registers exchange contents.

35. How do you read data over SPI? Send a dummy byte (0x00) to generate the clocks; the byte received during that transfer is the data.

36. Explain CPOL and CPHA. CPOL sets the idle level of SCK; CPHA selects whether data is sampled on the first or second edge. Mode 0 = idle low, sample on the rising edge.

37. Which SPI modes does the 25LC256 support, and which did you use? Modes 0,0 and 1,1; we use mode 0.

38. What is the SPI clock in your design? 375 kHz = 3 MHz / 8; one byte takes about 21.3 µs.

39. Why can't S0SPCCR be less than 8? The LPC2138 SPI0 hardware requires an even value of at least 8 in master mode — that is the maximum clock it supports relative to PCLK.

40. How does the firmware know a transfer is complete? It polls the SPIF bit in S0SPSR; SPIF is cleared by reading S0SPSR and then accessing S0SPDR.

41. What is a mode fault? In master mode, if the SSEL0 input becomes active, the controller sets MODF and leaves master mode (multi-master protection).

42. Why didn't you use the hardware SSEL0 pin as chip select? It's an input in master mode, and the EEPROM needs CS held low across multi-byte commands, so a GPIO pin is used instead.

43. What happens if SPI never completes? After 100 000 polls `spi_transfer` returns error 1; the motor is never started and the LED blinks.

44. SPI vs I²C — when would you use each? SPI for speed and simplicity with few devices; I²C to save pins with many devices on two wires, accepting lower speed and protocol overhead.

45. Does SPI have error detection? No acknowledge or CRC at the protocol level; errors must be detected at a higher layer — here, by the WEL check, the status sanity check and the profile checksum.

46. What is the difference between SPI0 and SSP on the LPC2138? SSP is a more capable synchronous serial port with an 8-frame FIFO, variable frame size and multiple formats (SPI, SSI, Microwire); SPI0 is simpler, with a single-byte buffer.

D. EEPROM (47–60)

47. How does an EEPROM store a bit? As charge on the floating gate of a MOSFET, changed by Fowler–Nordheim tunnelling under high voltage from an internal charge pump; the charge shifts the threshold voltage, which is sensed on read.

48. EEPROM vs flash? Both use floating gates; EEPROM is byte-erasable, flash erases whole sectors (denser, cheaper, needs sector management).

49. Size and organisation of the 25LC256? 256 Kbit = 32 768 × 8 bytes, in 64-byte pages, addressed with 16 bits (15 used).

50. Steps to write data? WREN (CS high afterwards) → confirm WEL → WRITE + address + data (≤ 64 bytes, within one page) → CS high starts the write → poll RDSR until WIP = 0.

51. What are WEL and WIP? WEL = write-enable latch (must be 1 before writing; cleared after each write); WIP = write in progress (1 while the internal cycle runs).

52. Why is WREN needed before every write? WEL is cleared automatically after every completed write, at power-up and by WRDI — a safety feature against accidental writes.

53. What is page wrap? Writing past the end of a 64-byte page wraps to the start of the same page and overwrites data; writes must be split at page boundaries, as `ee_write_block` does.

54. How long does a write take? Up to 5 ms (self-timed); reads are limited only by the SPI clock.

55. Why poll WIP instead of a fixed delay? It finishes as soon as the device is ready, doesn't depend on CPU speed, and detects a stuck device through a time-out.

56. What are WP and HOLD? WP protects the status register when WPEN = 1; HOLD pauses a transfer. Both are tied high (inactive).

57. What are the block-protect bits? BP1:BP0 protect none / the upper ¼ / the upper ½ / the whole array; the firmware refuses to continue if the whole array is protected.

58. What is EEPROM endurance, and is it a concern here? About 1 million writes per byte; this design writes only when the profile is invalid (normally once), so no.

59. How do you detect that the EEPROM is not connected? Status 0xFF (MISO floating or pulled high) → error 3. A real status can't be 0xFF because the unused bits read as 0.

60. Why check WEL after WREN? It proves the EEPROM received the command and that MISO returns real data; a stuck-low MISO would otherwise look like "write finished, no errors".

E. Data integrity (61–70)

61. How is your profile stored? At 0x0000: 'M', 'P', version, step count, 6 bytes per step (big-endian 16-bit times), then a checksum — 29 bytes for 4 steps.

62. Why not store the C struct directly? Padding, endianness and layout depend on the compiler; explicit serialisation defines the format exactly.

63. How is the checksum calculated? Two's complement of the byte sum, so the sum of all bytes including the checksum is 0 mod 256. For our profile the sum is 1721 → 185 → checksum

0x47.

64. What can the checksum miss? Errors that cancel out, byte swaps, and changes totalling a multiple of 256. A CRC catches these.

65. Explain CRC briefly. The data is treated as a polynomial and divided (in modulo-2 arithmetic) by a generator polynomial; the remainder is stored. It detects all burst errors up to its width and most multi-bit errors.

66. Why the magic bytes? To recognise a blank (0xFF) or foreign EEPROM before trusting the contents.

67. Why a version number? So new firmware can reject old data with a different meaning and write updated defaults.

68. Why range checks if the checksum already passed? Defence in depth: a valid checksum on wrong data (for example from buggy writing code) must still never drive the motor with impossible values.

69. Why read back after writing defaults? It verifies the write physically succeeded; the motor only runs data that came from the EEPROM.

70. What if power fails during a write? The page may be torn; the next boot detects a bad checksum and restores the defaults. A production design would use A/B copies with sequence numbers so a good copy always survives.

F. Timers, PWM and motor (71–86)

71. How does an LPC2138 timer count? The prescale counter counts PCLK cycles up to PR; then TC increments and the prescale counter resets — one TC tick every (PR + 1) PCLK cycles.

72. How did you get a 1 ms tick? Timer1 PR = 3 000 000/1000 − 1 = 2999.

73. How does Timer0 generate the PWM phase? PR = 299 gives 100 µs ticks; MR0 = 99 with reset-on-match makes TC count 0–99 repeatedly (10 ms period).

74. What is PWM, and what is duty cycle? A fixed-frequency on/off signal; duty = on-time / period. Average voltage = duty × supply.

75. Why PWM instead of reducing the voltage linearly? The switches are either fully on or fully off, so little power is lost as heat — high efficiency.

76. Explain your software PWM. In `pwm_service`, EN1 is high while T0TC < duty; it is called continuously from `delay_ms`, so it runs whenever the program waits.

77. Why 100 Hz? The analog motor and driver models in Proteus slow down with every switching edge; 3 kHz made the simulation run far slower than real time, while 100 Hz runs at 29 % CPU load. On hardware, 20 kHz or more is preferred.

78. How does the hardware PWM unit work? MR0 sets the period; in single-edge mode output n goes high at the period start and low at MRn; PWMLER latches new values at the next period to avoid glitches.

79. What is the relationship between PWM duty and motor speed? Approximately linear: steady speed ≈ (V_avg − I·R)/K_e, and V_avg = duty × V_S.

80. Why ramp the speed? At start, back-EMF is zero, so current ≈ V/R (stall current); ramping limits inrush current and mechanical shock.

81. Why ramp down and pause before reversing? A spinning motor's back-EMF adds to the reversed supply, giving up to about twice the stall current; stopping first protects the driver, the supply and the gearbox.

82. What is an H-bridge? Four switches around the motor; closing opposite corners applies the supply in either polarity. Closing both switches on one side (shoot-through) shorts the supply.

83. Explain the L293D pins you used. EN1 (PWM), IN1 and IN2 (direction), OUT1 and OUT2 (motor), VSS 5 V logic, VS 12 V motor supply, ground pins.

84. What are flyback diodes? Diodes that give the inductive motor current a path when the transistors turn off, clamping the voltage spike V = L·di/dt; the L293D has them built in.

85. Brake vs coast? Brake: both outputs at the same level, motor terminals shorted, fast stop. Coast: outputs off, the motor spins down freely. PWM on EN gives coast during the off-time.

86. What limits the L293D? About 600 mA per channel continuous, and the voltage drop of its bipolar output transistors (efficiency); MOSFET drivers are better for higher currents.

G. Embedded C and toolchain (87–95)

87. What does the start-up code do? It sets SP, copies `.data` from flash to RAM, zeroes `.bss`, and calls `main`.

88. What is in a linker script? Memory regions and section placement: vectors at 0, code and constants in flash, `.data` in RAM with its initial values stored in flash, `.bss` in RAM, plus boundary symbols for the start-up code.

89. VMA vs LMA? Run address vs load address; `.data` runs in RAM but is stored in flash.

90. What is the `.bss` section? Uninitialised (zero-initialised) globals; it takes no space in the image and is zeroed at start-up.

91. What does `volatile` do? It forces every read and write to actually occur, which is needed for hardware registers, delay loops and debug variables.

92. Why `-ffreestanding` and `-fno-builtin`? There is no hosted C library; the compiler must not assume one or insert library calls.

93. ELF vs HEX? ELF holds code, symbols and debug information (used for debugging and simulation); HEX is a text encoding of the raw program bytes for programmers, with a checksum per record.

94. What does `static` mean on a function or global? Internal linkage: the name is private to the file.

95. Why use the `U` suffix in `1U << 31`? It makes the constant unsigned; shifting into the sign bit of a signed int is undefined behaviour.

H. Testing and debugging (96–100)

96. How did you test error handling? In the emulator, by injecting a blank, corrupt and old-version EEPROM, an unresponsive EEPROM, a stuck MISO and a dead SPI clock; each produced the expected error code with the motor off.

97. How did you prove the EEPROM holds the right data? The Proteus memory window shows `4D 50 02 04 …` ending with checksum 0x47 — the exact encoded default profile.

98. Describe a bug you fixed. Any STAR story from Part 10; Case 2 (inconclusive test) and Case 5 (simulation speed) are the strongest.

99. Was it tested on real hardware? Not yet; it is verified in Proteus and in emulation. The next step is a board with an oscilloscope and a logic analyser.

100. What did you learn? Register-level peripheral programming, SPI and EEPROM protocols, designing for data integrity and fail-safe behaviour, the build pipeline from C to HEX, and evidence-based debugging.

PART 15 — GLOSSARY AND HONESTY NOTES

Glossary

Term	Meaning
Bare-metal	Firmware that runs directly on the hardware, with no operating system
Back-EMF	Voltage generated by a spinning motor, opposing the applied voltage
BSS	Section of zero-initialised global variables
Charge pump	On-chip circuit generating the high voltage for EEPROM programming
CPOL / CPHA	SPI clock polarity / phase
Duty cycle	Fraction of a PWM period during which the output is on
EABI	Embedded Application Binary Interface — calling and object-file conventions
ELF	Executable and Linkable Format
Endurance	Number of erase/write cycles a memory cell survives
Flyback diode	Diode that clamps the inductive voltage spike when a switch opens
H-bridge	Four-switch circuit that drives a motor in either direction
ISP	In-System Programming (through the boot loader over UART)
LMA / VMA	Load memory address / virtual (run-time) memory address
MODF	SPI mode-fault flag
Page write	EEPROM write of up to one page (64 bytes) in a single cycle
PCLK / CCLK	Peripheral clock / CPU clock
PWM	Pulse-width modulation
SPIF	SPI transfer-complete flag
Shoot-through	Both switches of one H-bridge leg on at once — a short circuit
Volatile	C qualifier forcing every access to memory to actually happen
WEL / WIP	EEPROM write-enable latch / write-in-progress bits

Honesty notes — read before every interview

- **It was a team project.** Say “we” for the whole project and “I” only for parts you personally did.
- **It is verified in simulation and emulation, not on hardware.** Say so confidently; it is a strength to know the difference.
- **AI assistance was used** for parts of the code, the emulator testing and the documentation. Be ready to explain every line yourself — Parts 3–8 cover all of it.
- **Quote only real numbers:** 29 % CPU load at 00:03:36 simulated, checksum 0x47, 9.9 s cycle, 5448-byte image, 375 kHz SPI.
- **If you don’t know an answer:** “I haven’t worked with that directly, but I would approach it by …” — never invent datasheet values.

Repository: github.com/Ashish181104/lpc2138-spi-eeprom-motor-controller